

Fourier Analysis Projects: Applications of Fourier Transformations and Fast Fourier Transformations in Signal Processing, PDEs, Imaging, and MRI Simulation.

James Rodney
May 2026

Abstract

This paper presents a collection of projects exploring Fourier analysis and its applications in signal processing, partial differential equations, image processing, and MRI reconstruction. Using primarily the Fast Fourier Transform (FFT), Short Time Fourier Transform (STFT), projects were implemented using C++ and MATLAB. Projects ranged from simple simulated signals, to image processing, to MRI reconstruction using the inverse 2D Fourier Transform from real k-space data. Additionally, a simulated MRI pipeline was constructed using first principles. These projects demonstrate remarkable diversity of Fourier Transform and its critical mathematical principle in broad range of scientific and engineering applications.

Acknowledgments

The author acknowledges guidance from Professor Brett Peterson and Professor Miguel Sanchez at Arizona Western College. Artificial intelligence tools were also used for coding assistance, debugging, and editing. All final implementations and interpretations were reviewed by the author.

Table of Contents

-Introduction (pp. 3-7)

Table of projects

FFT

- Denoising Simulated Data (pp. 8-9)
- Audio Denoising (pp. 10-11)
- Find Derivatives Using FFT (pp. 12-13)

STFT

- Windowing FFT – Short Time Fourier Transform (pp. 14-16)
- Windowing Audio Denoising (pp. 17)
- Fourier Spectrogram (pp. 18-19)

PDEs

- 1D Heat Equation (pp. 20)
- 2D Heat Equation (pp. 21-22)
- 1D Wave Equation (pp. 23-24)
- 2D Wave Equation (pp. 25-27)
- 2D Damped Wave Equation (pp. 28-32)

Imaging

- Fourier Image Denoising using Low Pass Filter (pp. 33-34)
- STFT Image Processing (pp. 35-36)
- NYU fastMRI MRI Image Processing (pp. 36-40)
- Simulated MRI (pp. 41-43)

-Discussion (pp. 44-46)

-Conclusion (pp. 47)

-References (pp. 48)

Introduction

This paper documents a series of projects related to the Fourier transformation. The goal of these projects was to understand what the Fourier Transformation is as well as its possible applications.

The Fourier Transformation was originally developed by physicist Joseph Fourier to study heat conduction in objects. The Fourier Transformation comes from the concept of the Fourier Series, which states that any periodic function can be expressed as an infinite sum of sinusoids of increasing frequencies. Essentially, the full Fourier transform, defined as:

$$F(k) = \int_{-\infty}^{\infty} f(t)e^{-2\pi ikt} dt$$

It takes a function of time and outputs another function of frequencies that tells the strength of that frequency within the original function, i.e. how much that frequency contributes to the original function. It can be thought of as a mathematical prism, just as a prism takes a ray of light and splits it into what colors make it up, the Fourier transform takes a function (or signal) and splits it into its individual frequencies. For example, without going into the derivation for simplicity's sake, the Fourier Transform of a function $g(t) = \sin(50\pi \cdot t) + 0.5\cos(30\pi \cdot t)$:

$$\begin{aligned} G(\omega) &= \int_{-\infty}^{\infty} (\sin(50\pi \cdot t) + 0.5\cos(30\pi \cdot t))e^{-2\pi i\omega t} dt \\ \Rightarrow \int_{-\infty}^{\infty} \sin(50\pi \cdot t) e^{-2\pi i\omega t} dt &+ 0.5 \int_{-\infty}^{\infty} \cos(30\pi \cdot t) e^{-2\pi i\omega t} dt \end{aligned}$$

Identities:

$$\begin{aligned} F\{\cos(2\pi f_0)\} &= [\delta(f - f_0) + \delta(f + f_0)] \\ F\{\sin(2\pi f_0)\} &= -i[\delta(f - f_0) - \delta(f + f_0)] \end{aligned}$$

$$\Rightarrow G(\omega) = 0.5[\delta(\omega - 15) + \delta(\omega + 15)] - i[\delta(\omega - 25) - \delta(\omega + 25)]$$

Graphing the Fourier Transform of $g(t)$, would give a spike at $\omega = \pm 15$ and $\omega = \pm 25$ in the complex plane.

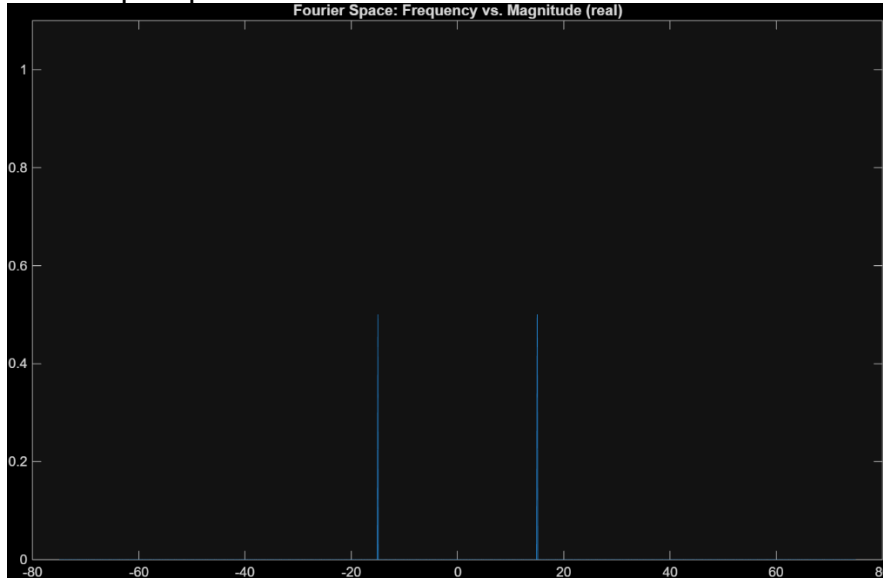


Figure 1. Fourier Space, frequency vs amplitude in real components

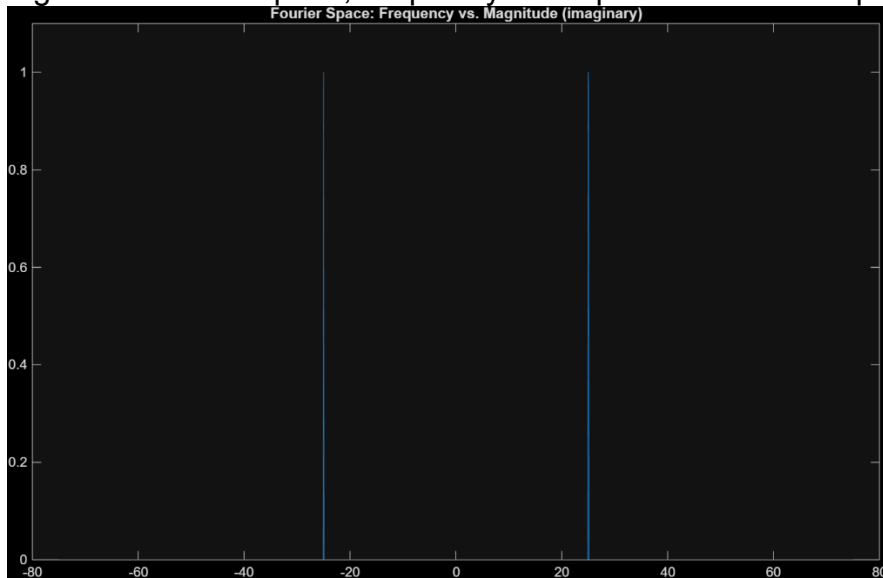


Figure 2. Fourier Space, frequency vs amplitude in imaginary components.

The double peak at plus/minus is an artifact of the Fourier Transform that comes from Euler's formula that cosine/sine decompose into 2 complex exponentials. This symmetry comes into play later with the Nyquist frequency.

Another interesting Fourier transform is that the transform of the Gaussian is another Gaussian:

$$F\{g(k)\} = \int_{-\infty}^{\infty} e^{-x^2} e^{-2\pi i k x} dx = \sqrt{\pi} e^{-\pi^2 k^2}$$

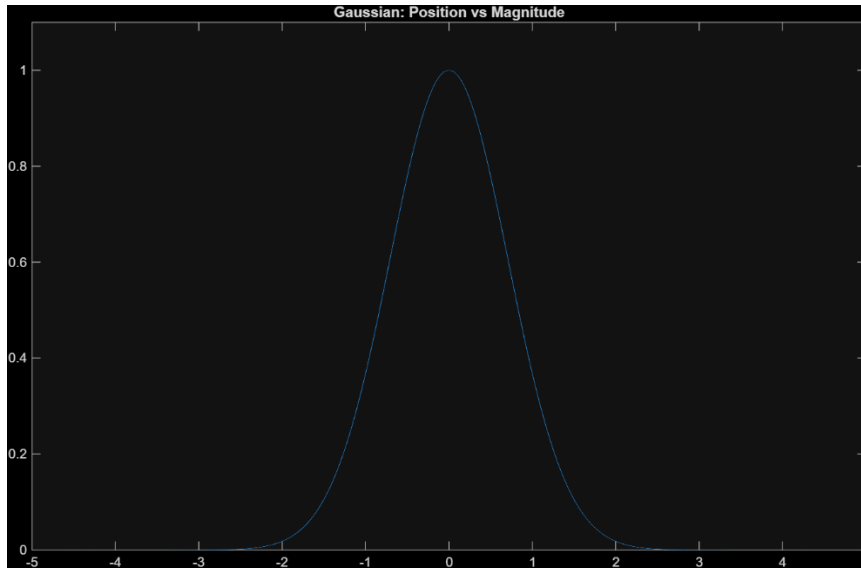


Figure 3. Graph of the Gaussian, position vs. magnitude.

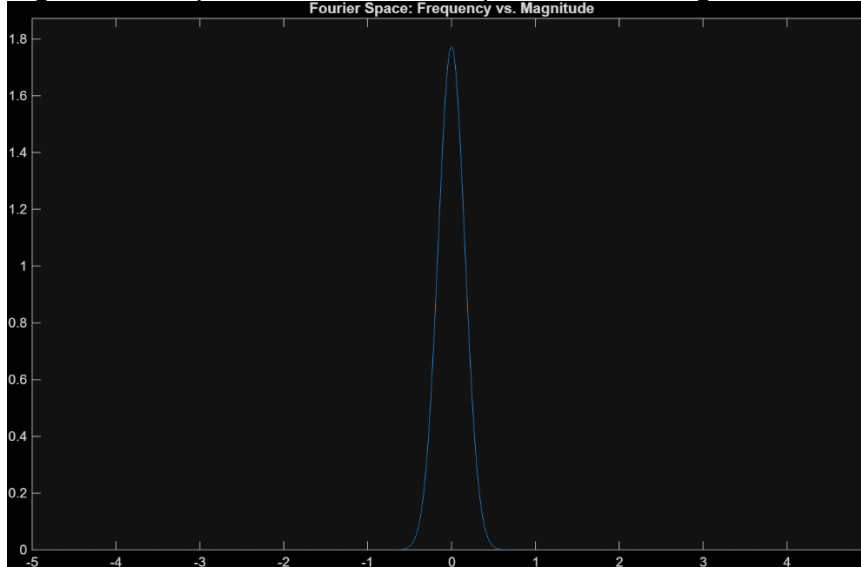


Figure 4. Graph of the Fourier Transformation of the Gaussian; another Gaussian.

Now that a basic understanding of the Fourier Transform has been established, how does it apply to discrete datasets? This is where the Discrete Fourier Transform (DFT) comes in. The DFT is defined as:

$$F_n = \sum_{k=0}^{N-1} f_k e^{-2\pi i n k / N}$$

Meaning that if there are n number of samples there will be n number of “frequency bins.” The resolution of these frequency bins can be defined as:

$$\Delta f = \frac{fs}{N} = \frac{1}{T}$$

fs: Sampling rate

N: number of samples

T: total time sampled

If a frequency falls between the two frequency bins, then the frequency's amplitude is split into the adjacent bins, this is known as spectral leakage.

Because the Fourier transform of a real-value signal's is conjugate symmetric.

Magnitude spectrum is symmetric about the y axis, while the phase spectrum is anti-symmetric. Essentially roughly half of the bins correspond to negative frequencies, which are redundant, and are often left out in graphs. It also means that the highest frequency that can be detected is half the number of samples multiplied by Δf , or:

$$f_{nyquist} = \frac{N}{2} \times \frac{1}{T} = \frac{fs}{2}$$

For example, a signal sampled for 1 second at a sampling rate of 512hz the highest frequency that can be detected is 256hz. This is known as the Nyquist Frequency, defined as half the sampling rate. Which also plays into the Nyquist Theorem which states that a continuous signal can be perfectly reconstructed iff it is sampled at a sampling rate greater than twice the highest frequency component.

Also, depending on the program, the way that the frequency bins are organized within the Fourier Transform matrix can vary. Such as with MATLAB, often times the ShiftFFT is used, which just reorganizes the matrix into a more readable format.

The DFT has one problem, in most real-world applications, it is far too slow. The number of computations required is N^2 , meaning for a relatively slow sampling rate of 1000hz, sampled for 1 second it would take 1,000,000 computations to calculate. The creatively named Fast Fourier Transform (FFT), is an algorithm used to solve this problem and calculate the DFT. It takes the number of computations required from N^2 down to $N \log_2 N$. It was first developed in the 1960s by John Tukey and James W. Cooley to help detect nuclear detonations during the Cold War. Though, some argue that Gauss first came up with the idea in an unpublished document written in 1805.

Despite coding the FFT from scratch in C++, an in-depth analysis of the Tukey-Cooley algorithm is beyond the scope of this paper. However, the basic idea behind FFT is that it takes advantage of the symmetries of the Fourier Transform. To do this it splits the dataset in two by putting all the even indexes in one subset and all the odds in another subset, it repeats recursively until each subset only has 2 samples, then it performs the DFT on the odds samples and adds it to the even samples and then it works its way back up the chain, yielding a 100% accurate DFT with significantly less computational cost.

The FFT does have a limitation that the dataset must be a power of 2. However, to get around this limitation, the dataset will get "ghost samples" and will be padded with zeros until the number of samples within the dataset is a power of 2.

Brief note on notation: Throughout this paper, ω denotes angular frequency with respect to time (rad/s), κ denotes angular spatial frequency (rad/m), and f denotes frequency in cycles per second (Hz).

Denoising Simulated Data Project

Language: C++ (V1), MATLAB (V2)

Credit: Steve Brunton's Fourier Analysis Video Series

Theory

This program was the first in the series as such, it was the first test of my Fast Fourier Transform program. The idea was to simulate a noisy digital signal by making a function of combination of 2 different sine waves and then adding random noise to the dataset:

$$g(t) = \sin(2\pi * f_1 * t) + \sin(2\pi * f_2 * t) + noise$$

On the simulated signal function data points were sampled at a set sampling rate. Then a Fast Fourier Transformation (FFT) was applied. The result is a 1D matrix of frequencies denoted as $\hat{f}$ where each entry is a "bin" that corresponds to a different equally spaced frequency, inside of the bin is a number that represents how much of that frequency is present in the original signal.

The expected result of the FFT for a function composed of 2 sine waves with frequencies f_1 and f_2 would a spike in the frequency bin that corresponds to f_1 and another spike at the bin corresponding to f_2 . Using this with the fact that all other frequencies will be contributing a very small amount due to the only other source being random noise, we can set all other bins that are not the f_1 and f_2 bins to 0. Then taking the Inverse Fast Fourier Transform of the compressed $\hat{f}$, the original signal minus the noise should be constructed.

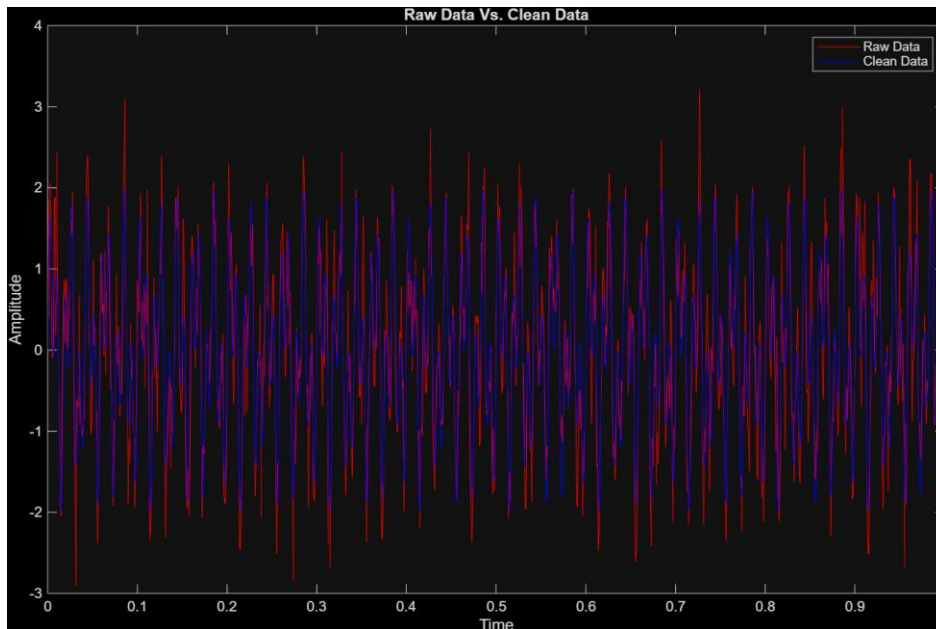


Figure 1. Raw signal with expected clean signal overlayed.

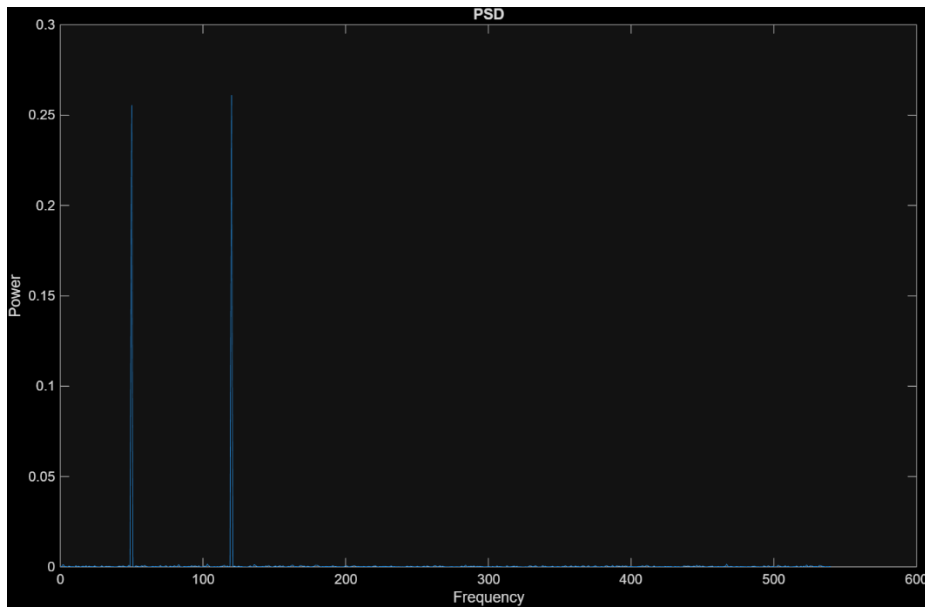


Figure 2. Power Spectrum Density graph with the 2 peaks 50hz and 120hz.

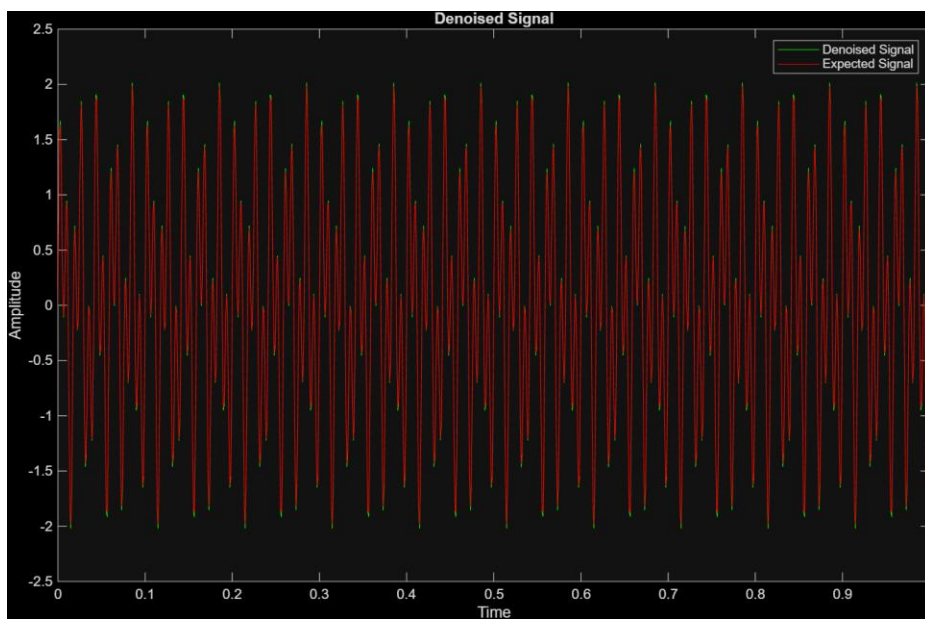


Figure 3. Denoised data with the expected clean signal overlayed.

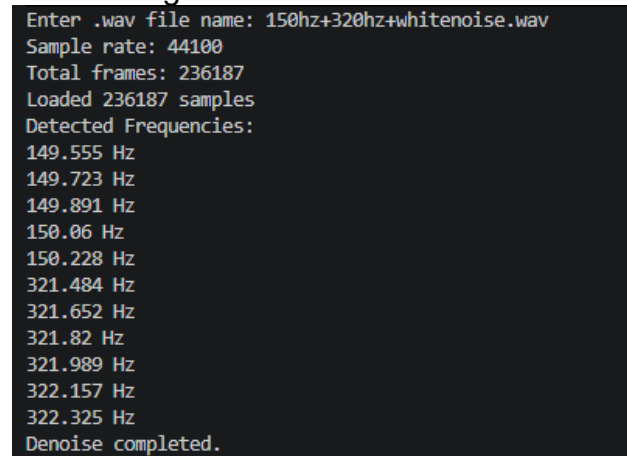
Audio Denoising

Language: C++

Theory

This program is a real world application of the previous Denoising Simulated Data Program. Instead of synthesizing the data it takes an audio file that is a mix of a constant frequency with noise added it. The program utilizes the Libsndfile library to read the .wav file, extracting amplitude vs time data. It then goes through the previously described process and then writes the clean data to a new .wav file.

Figure 4. shows the result of running an audio file called “150hz+350hz+whitenoise.wav” through the program. It shows the sampling rate of the file, total number of samples, and the detected frequencies. It shows that more than just 150hz and 320hz were detected, this is due to spectral leakage, an artifact of the FFT. Spectral Leakage occurs when the detected frequencies in the FFT does not have an exact bin to fit the frequency, so the detected frequency gets distributed into the surrounding bins.



```
Enter .wav file name: 150hz+320hz+whitenoise.wav
Sample rate: 44100
Total frames: 236187
Loaded 236187 samples
Detected Frequencies:
149.555 Hz
149.723 Hz
149.891 Hz
150.06 Hz
150.228 Hz
321.484 Hz
321.652 Hz
321.82 Hz
321.989 Hz
322.157 Hz
322.325 Hz
Denoise completed.
```

Figure 4. Consol output displaying the sample rate of the file and number of samples, with the detected frequencies displayed below.

Fig 5 shows the results of the audio denoiser visualized using Audacity. The top audio track shows the raw audio signal, playing the file sounded like a medium pitched hum with static playing in the background. The bottom audio track shows the denoised audio signal, playing the audio sounded like the same medium pitched hum but without the static in the background. Looking at the denoised audio track, it appears that the amplitude has a little bit of oscillation, especially at the ends, this is likely due to the previously mentioned spectral leakage.

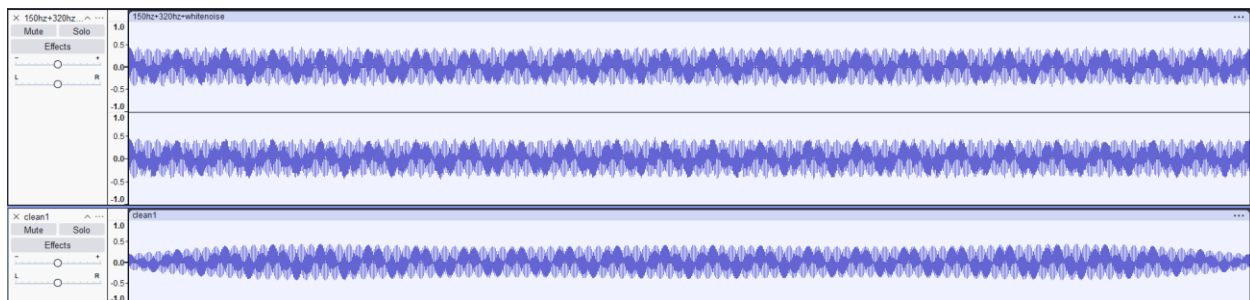


Figure 5. Audio in waveform, the top 2 tracks display the waveform of the original audio, the bottom track displays the waveform of the denoised audio.

Finding Derivatives Using FFT

Language: MATLAB

Credit: Steve Brunton's Fourier Analysis Video Series

Theory

This program computes discrete derivatives using the FFT and compares the result to other methods. Finding the Fourier Transformation of any derivative $\frac{dy}{dx}$ using the integral definition:

$$F\left\{\frac{dy}{dx}\right\} = \int_{-\infty}^{\infty} \frac{dy}{dx} e^{-i\kappa x} dx$$

Using integration by parts:

$$\begin{aligned} u &= e^{-i\kappa x} \\ du &= -i\kappa e^{-i\kappa x} dx \\ dv &= \frac{dy}{dx} \\ v &= y \\ \Rightarrow [ye^{-i\kappa x}]_{-\infty}^{\infty} - \int_{-\infty}^{\infty} (y)(-i\kappa e^{-i\kappa x}) dx \end{aligned}$$

For the Fourier Transform to apply $\lim_{x \rightarrow \pm\infty} y = 0$, and the range for $e^{-i\kappa x}$ is

$[-1,1]$, therefore $[ye^{-i\kappa x}]_{-\infty}^{\infty} = 0$. Additionally, $-i\kappa$ is a constant so it can be pulled out of the integral. Therefore:

$$\Rightarrow i\kappa \int_{-\infty}^{\infty} ye^{-i\kappa x} dx$$

$\int_{-\infty}^{\infty} ye^{-i\kappa x} dx$ is the definition of the Fourier Transform of y , therefore:

$$F\left\{\frac{dy}{dx}\right\} = i\kappa F\{y\}$$

Using this information the derivative of a discrete set can be calculated by applying the FFT and multiplying by $i\kappa$ and inversing back to the spatial domain. The expected result is that the Fourier method of calculating derivatives requires significantly less sample points compared to traditional methods.

Figure 6. Shows the result calculating the derivatives of a function $f = e^{-x^2/25} \cos(x)$ by symbolically finding the derivative (exact), using the Fourier transform, and finding the slope $\frac{\Delta y}{\Delta x}$ (numerical).

Looking at figure 6. It is apparent that the FFT method is significantly more efficient the finding the slope. Looking at the bottom figure it can be seen that the percent error drop to nearly zero after about 16 samples while the numerical method still yields around a 65% error.

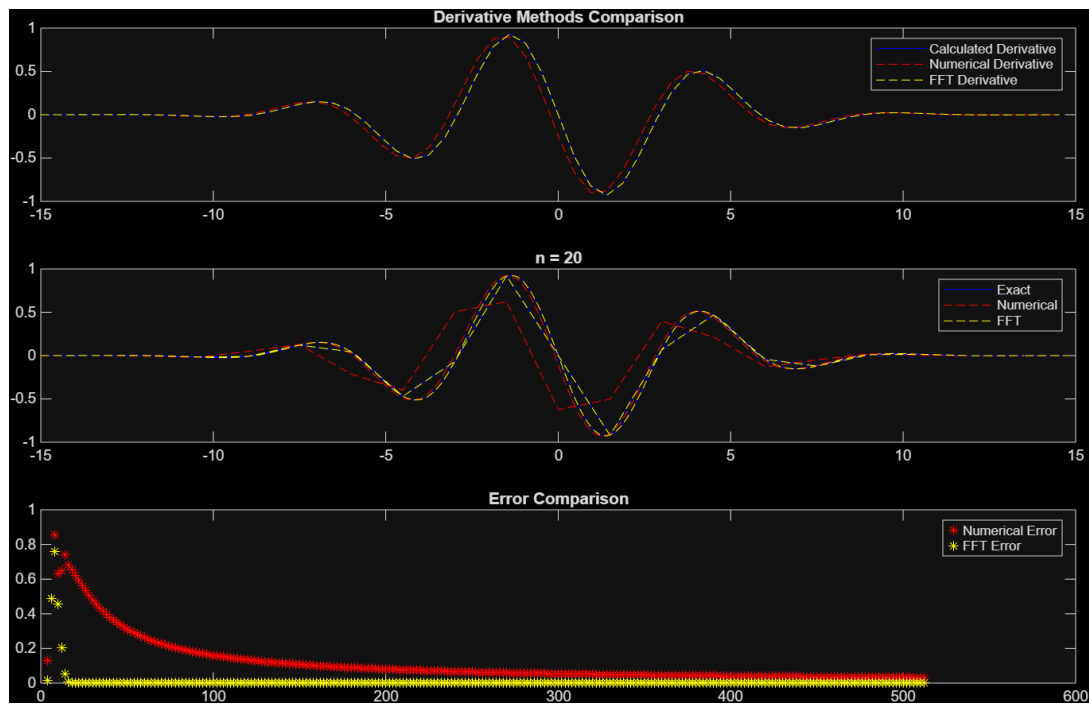


Figure 6. Displays 3 graphs displaying the numerical derivative vs FFT Derivative vs exact derivative.

Windowing FFT (Short Time Fourier Transform)

Language: C++

Theory

Using the full Fourier Transform gives all the frequencies present throughout the whole signal, however when those frequencies occur is mystery. In other words, it gives 100% frequency resolution, but no time resolution. What if one wanted to know what frequencies are present at what times? How could one gain some time resolution back? To solve this problem we can divide the signal into several overlapping “windows.”

Instead of performing the FFT over the entire signal, it is applied to each of the windows. This way it has a frequency resolution $\Delta f = \frac{\text{Sampling Rate}}{\text{Num of Samples}}$ and a time resolution $\Delta t = \text{Length of Window}$.

For the Fourier Transformation to work effectively the signal must be perfectly periodic in the window. To make each window periodic, and reducing spectral leakage, a windowing function is applied to the windowed signal. For a windowed signal, $S(t)$ and a windowing function $W(t)$, the windowed signal is calculated by:

$$S_w(t) = S(t) \times W(t)$$

For this project the Hann Window $W(t) = \frac{1}{2}[1 - \cos(2\pi t)]$ was chosen because of its property of if two Hann windows are overlapped by 50% every point where the windows overlap adds to 1. Essentially, using the Hann window eliminates the need to de-window at the end.

To simulate a clean signal a piecewise function is used:

$$f(t) = \begin{cases} \sin(2\pi * f_1 * t) + \sin(2\pi * f_2 * t), & t < T/2 \\ \sin(2\pi * f_3 * t) + \sin(2\pi * f_4 * t), & t > T/2 \end{cases}$$

Noise was then added:

$$S(t) = f(t) + \text{noise}$$

The number of sample points per window is set by the user. Zeros are added to the end of the dataset until a whole number of overlapping windows can fit into the dataset. Half the window sample length of zeros is added at the beginning and the end, to ensure that for every sample in the dataset is in 2 different windows. The dataset is then divided into the overlapping windows with overlap of 50%. A FFT is then applied to each window. Several things could be done while in the frequency domain of each of the windows, such as displaying the prominent frequencies within each step. Or for the case of this program, all non-prominent frequencies were set to 0 and an IFFT was performed and each window was added back to their original index positions, then the dataset was trimmed back to the original size of the dataset to construct a reconstructed denoised signal.

Comparing the result of the STFT in Figure 9. With the clean data in figure 8. It is clear that the amplitudes have some discrepancies, however it appears that the STFT preserves the frequencies fairly accurately with the exception of the transition region where the signal switches from the first function of the piece wise to the second.

Comparing to the FFT in Figure 10. The amplitudes are more accurate; however the frequencies appear completely differently the clean signal.

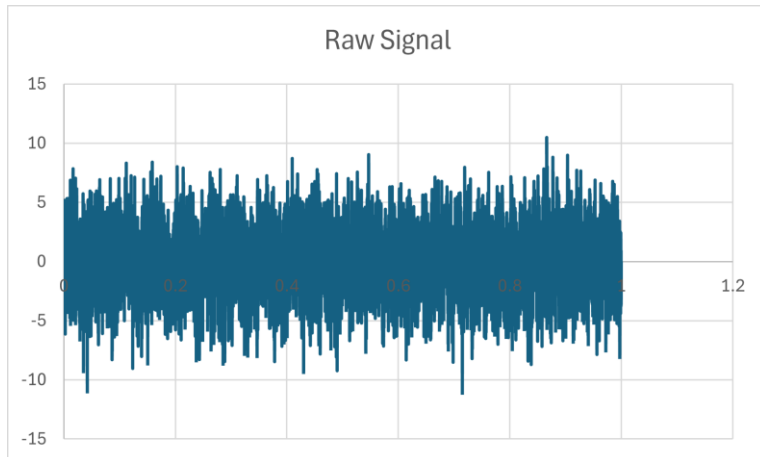


Figure 7. Displays the raw signal with $f_1 = 20$, $f_2 = 450$, $f_3 = 50$, and $f_4 = 120$

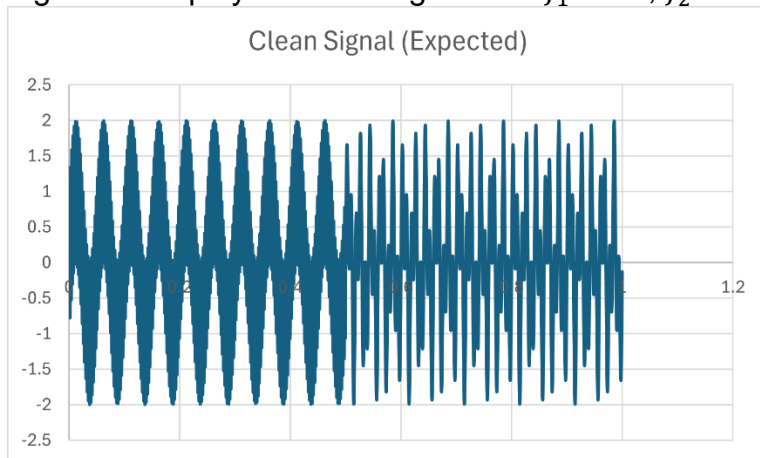


Figure 8. Expected graph of the raw signal without noise

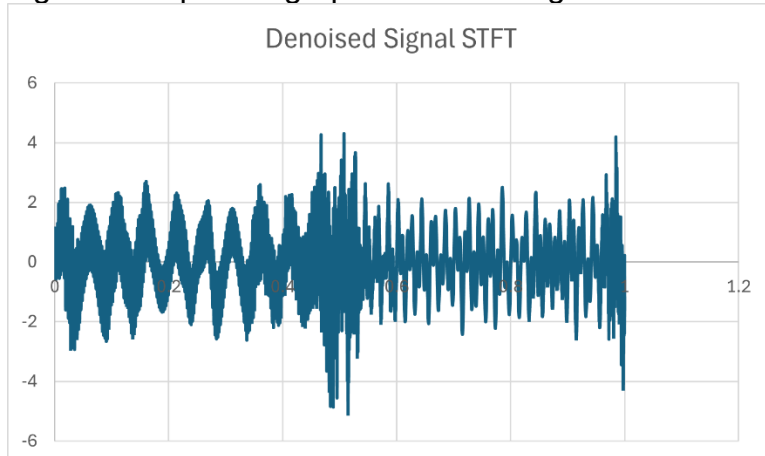


Figure 9. Denoised raw signal using STFT

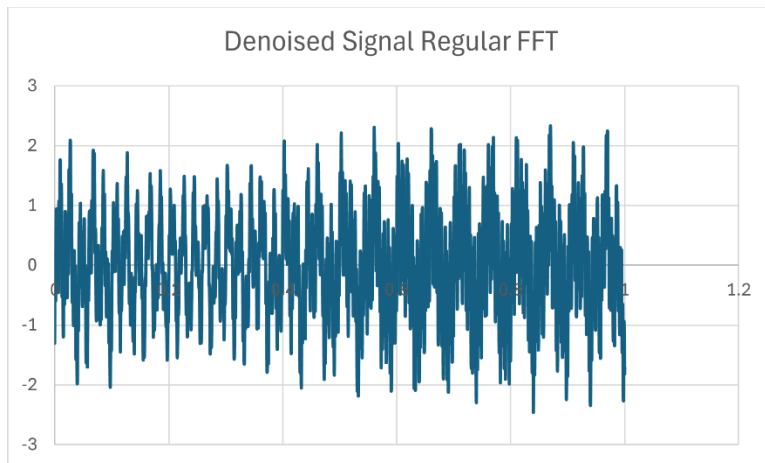


Figure 10. Denoised raw signal using regular FFT

Windowing Audio Denoiser (Short Time Fourier Transform)

Language: C++

Theory

This program is a real world application of the STFT. Similar to the regular FFT denoiser mentioned above, this program takes an audio file and extracts a vector of samples. The STFT is then applied to the samples. The least prominent frequencies in each window are set to 0. The samples of each window is inversed back and compiled to a clean vector. The vector is then trimmed back to the original size and exported to a clean audio file.

Figure 11. shows the waveform of the raw audio signal, it starts at 50hz and then transitions to 560hz with the 2 frequencies overlapping briefly about halfway through, white noise is added throughout the whole signal. Figure 12. shows waveform of the STFT denoised signal, and figure 13. Shows the regular FFT denoised signal. Comparing the signals, the STFT preserves the frequencies and overall shape extremely well, only losing some amplitude. Playing the STFT audio sounds like the original signal minus the noise, except slightly quieter. The FFT appears to struggle with preserving the frequencies at each time step, it is especially apparent in the transition zone, the signal appears to get louder and fade into the 560hz frequencies. However, no white noise could be observed when playing the audio track.

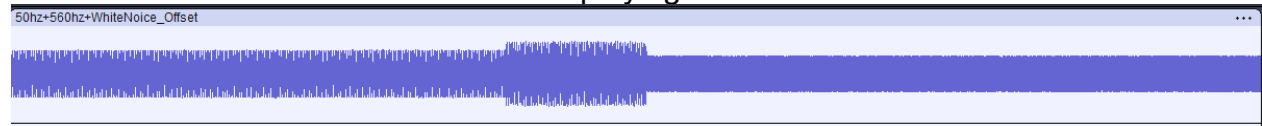


Figure 11. Audio waveform of 50hz switching to 560hz midway with overlap, with white noise added.



Figure 12. Audio waveform of denoised audio using STFT.

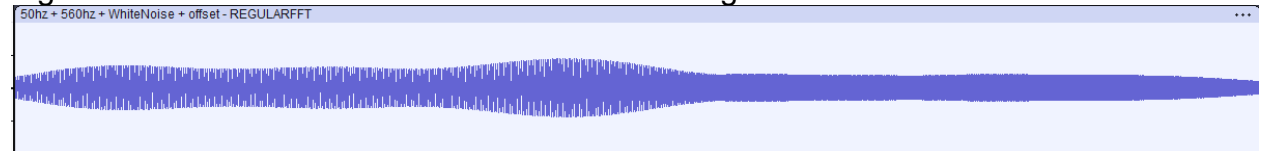


Figure 13. Audio waveform of denoised audio using regular FFT.

Fourier Spectrogram (Short Time Fourier Transform)

Language: MATLAB

Reference: Steve Brunton's Fourier Analysis Video Series

Theory

A spectrogram is used to visualize what frequencies are present at what times. A spectrogram can be constructed using the STFT. Essentially, each window is plotted with the y axis being frequency and the x axis being time, and the z axis or in this case, color, being the power of each frequency.

Figure 14. shows the raw signal; a cubic chirp with noise added. The signal was sampled at 1000hz with a total runtime of 2 seconds. Each window was set to a length of 128 samples. For an STFT this would give a frequency resolution of $\frac{\text{sampling rate}}{\text{window size}} = 7.8125\text{hz}$ and a time resolution of $\frac{\text{window size}}{\text{sampling rate}} = 0.1280$ seconds.

Figure 15 shows another method of displaying frequency data, a power spectrum density plot (PSD). A PSD is used to visualize how much each frequency is contributing to the final signal. For this signal, the PSD displays what is to be expected of a cubic chirp, the low frequencies are the ones with the highest power, because the chirp signal "spent" the most amount of time in the low frequencies.

Figure 16 displays the spectrogram of the signal. With the spectrogram, it can show not only how much each frequency is contributing, but also when each frequency is contributing. the most the spectrogram displays what would be expected of a cubic chirp, frequency starts low and gradually increases frequency following what appears to be a cubic curve.

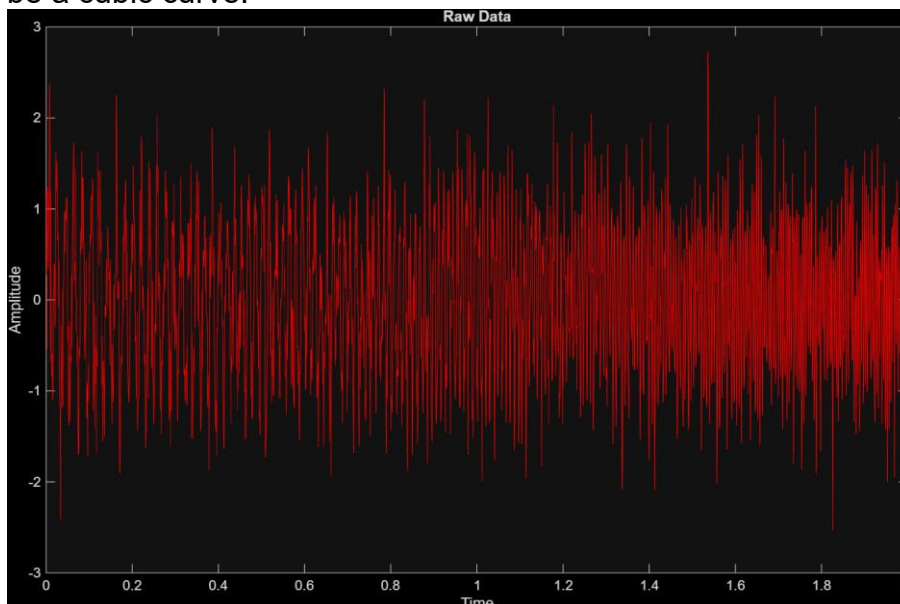


Figure 14. Time vs Amplitude graph of a cubic chirp with random noise.

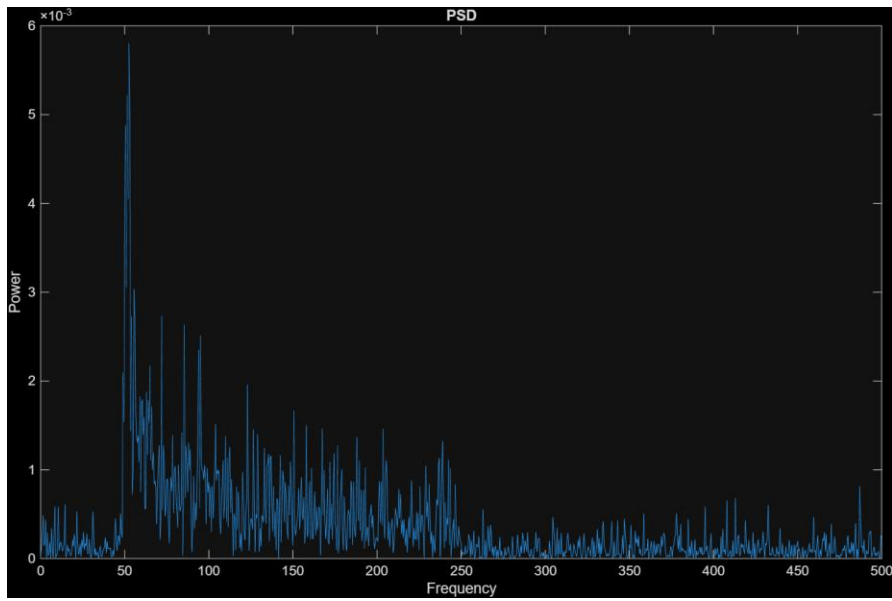


Figure 15. Power Spectrum Density of the signal.

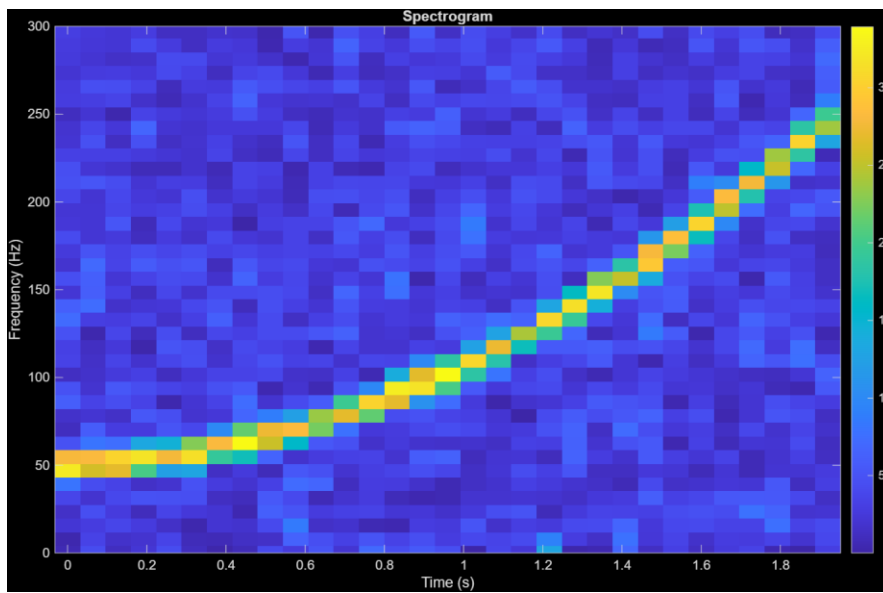


Figure 16. Calculated spectrogram of the signal using STFT.

1D Heat Equation

Language: MATLAB

Credit: Steve Brunton's Fourier Analysis Video Series

Theory

This project uses the Fourier Transform to solve the 1-dimensional heat equation:

$$\frac{\partial u}{\partial t} = \alpha \frac{\partial^2 u}{\partial x^2}$$

As previously shown, when taking the Fourier of a derivative it turns into multiplication by $i\omega$. Thus, taking the Fourier Transform of $\frac{\partial^2 u}{\partial x^2}$ with respect to x twice gives,

$$F\left\{\frac{\partial^2 u}{\partial x^2}\right\} = -\kappa^2 \hat{u}$$

Where $\hat{u}$ is a vector of decoupled ODEs, one for each κ_i . Thus, turning the heat equation into an ordinary differential equation:

$$\frac{\partial \hat{u}}{\partial t} = -\alpha \kappa^2 \hat{u}$$

Solving the ODE gives:

$$\hat{u} = C e^{-\alpha \kappa^2 t}$$

Applying initial conditions:

$$\begin{aligned} u(x, 0) &= f(x) \\ \xrightarrow{\text{Fourier}} \hat{u}(\kappa, 0) &= \hat{f}(\kappa) = \hat{u}_0 \end{aligned}$$

Gives:

$$\hat{u} = \hat{u}_0 e^{-\alpha \kappa^2 t}$$

Applying the inverse Fourier transform would give the solutions to the equation. This program will be using discrete values to solve, therefore the FFT and IFFT will be used to solve the equation, so the solution will be left as:

$$u(x, t) = F^{-1}(\hat{u}_0 e^{-\alpha \kappa^2 t})$$

Figure 17 shows how heat will diffuse as time increments by 50 seconds. The first slice shows the initial conditions of the system.

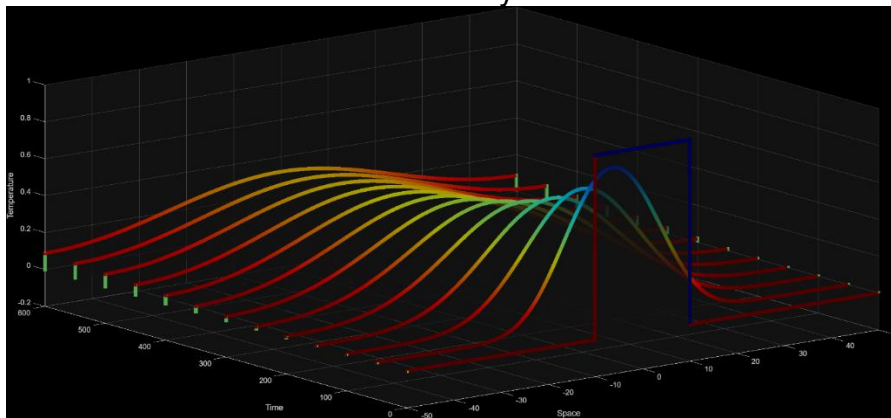


Figure 17. Waterfall graph shows how heat diffuses with initial conditions of a square wave.

2D Heat Equation

Language: MATLAB

Theory

This program solves the 2-dimensional heat equation:

$$\frac{\partial u}{\partial t} = \alpha \nabla^2 u = \alpha \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

Applying the Fourier, turns the PDE into an ODE:

$$\frac{\partial \hat{u}}{\partial t} = -\alpha(\kappa_x^2 + \kappa_y^2) \hat{u}$$

Solving the ODE with initial conditions, gives:

$$\hat{u}(\kappa_x, \kappa_y, t) = \hat{u}_0(\kappa_x, \kappa_y) e^{-\alpha(\kappa_x^2 + \kappa_y^2)t}$$

Thus:

$$u(x, y, t) = F^{-1}\{\hat{u}_0(\kappa_x, \kappa_y) e^{-\alpha(\kappa_x^2 + \kappa_y^2)t}\}$$

Figure 18 shows the initial conditions of the surface. And Figures 19 and 20 show how the heat diffuses across the surface.

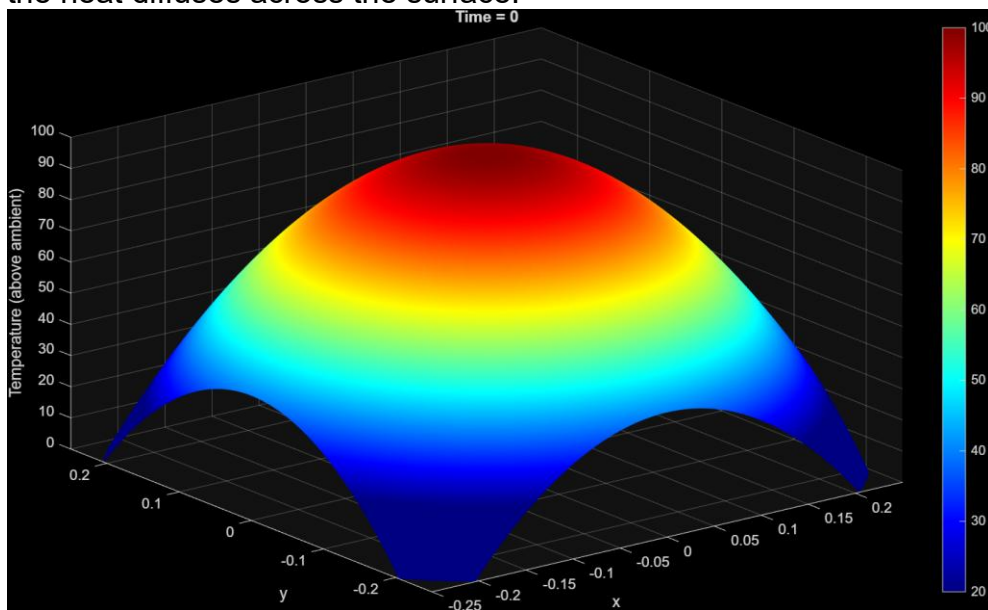


Figure 18. Shows the temperature of a plane at t=0 with initial conditions of a 2d gaussian.

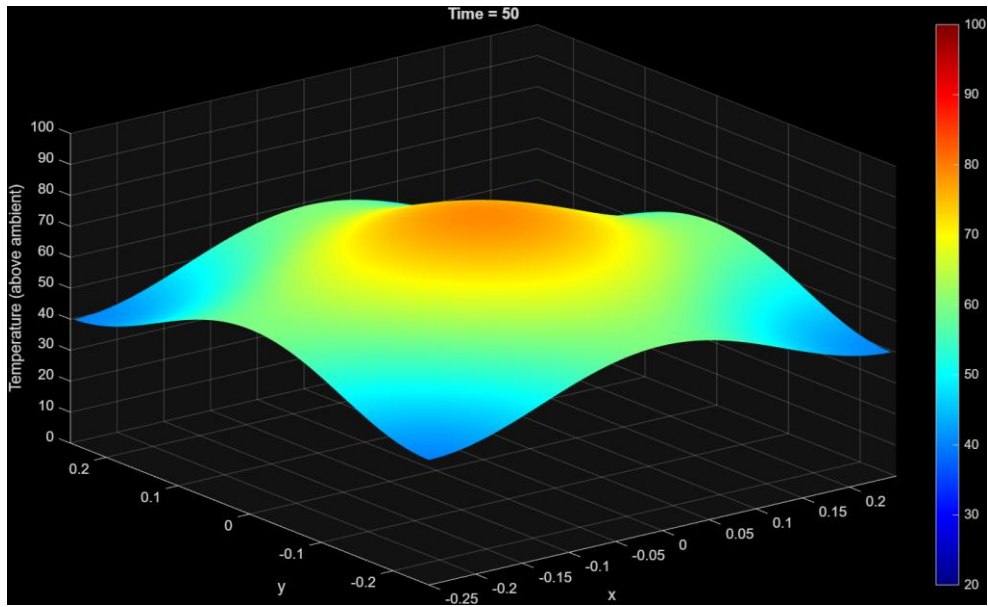


Figure 19. Shows the temperature of a plane at $t=50$ with initial conditions of a 2d gaussian.

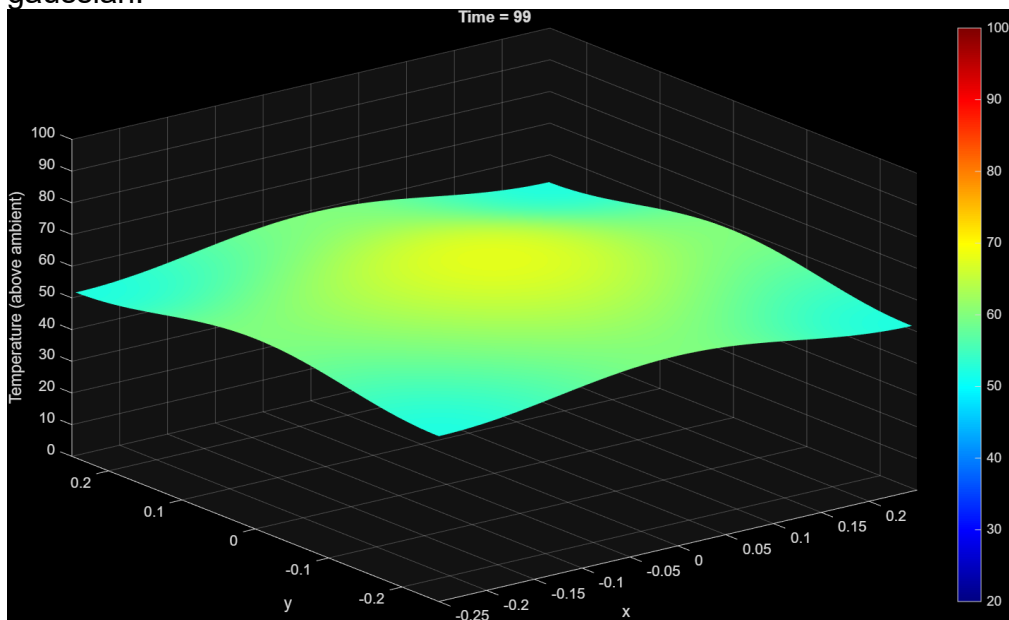


Figure 20. Shows the temperature of a plane at $t=99$ with initial conditions of a 2d gaussian.

1D Wave Equation

Language: MATLAB

Theory

This program uses the Fourier Transform to solve the 1-dimensional wave equation:

$$\frac{\partial^2 u}{\partial t^2} = v^2 \frac{\partial^2 u}{\partial x^2}$$

Applying the Fourier Transform with respect to x , transforming the second-order linear PDE into a second-order linear ODE:

$$\frac{\partial^2 \hat{u}}{\partial t^2} = -v^2 \kappa^2 \hat{u}$$

Solving the ODE using eigen values gives the eigen values:

$$\lambda = \pm v\kappa i$$

With eigenvectors:

$$\begin{aligned}\hat{u}_1 &= v\kappa i \\ \hat{u}_2 &= -v\kappa i\end{aligned}$$

$$\Rightarrow \hat{u}(\kappa, t) = A(\kappa) \cos(v\kappa t) + B(\kappa) \sin(v\kappa t)$$

Solving with the initial conditions:

$$\begin{aligned}u(x, 0) &= f(x) \\ \frac{\partial u}{\partial t} &= g(x)\end{aligned}$$

Gives the spectral domain solution:

$$\hat{u}(\kappa, t) = \hat{f}(\kappa) \cos(v\kappa t) + \frac{\hat{g}(\kappa)}{v\kappa} \sin(v\kappa t)$$

This program will be using discrete values with the FFT and IFFT, so the solution will be left as:

$$u(x, t) = F^{-1}\{\hat{f}(\kappa) \cos(v\kappa t) + \frac{\hat{g}(\kappa)}{v\kappa} \sin(v\kappa t)\}$$

Figure 21 shows a gaussian pulse ($f(x) = e^{-x^2}$) with initial velocity of $g(x) = 0$ travelling through space with respect to time. The pulse first splits into 2 smaller waves that travel in opposite directions until hitting the boundaries, where they reappear on the opposite side, appearing to bounce back and eventually combine back into the original gaussian pulse.

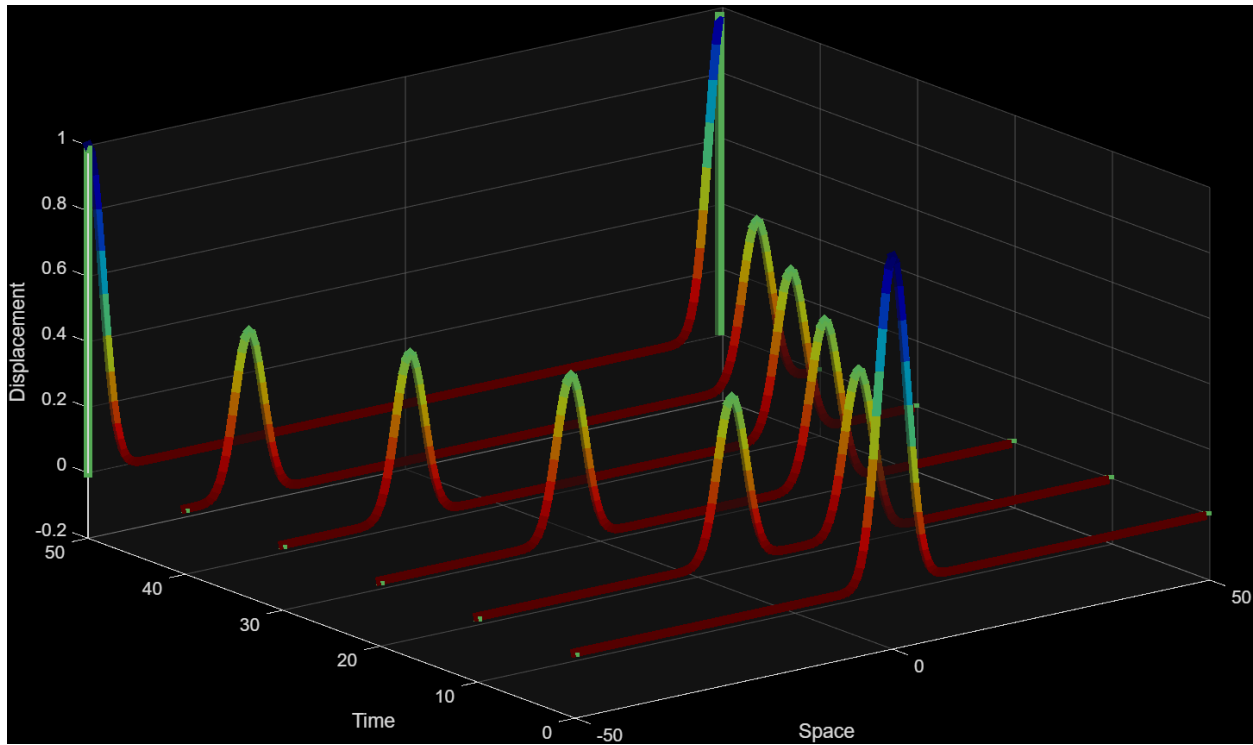


Figure 21. Waterfall graph that shows how a gaussian wave travels in space with respect to time.

2D Wave Equation

Language: MATLAB

Theory

This program uses the Fourier Transform to solve the 2-dimensional heat equation

$$\frac{\partial^2 u}{\partial t^2} = v^2 \nabla^2 u = v^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

Taking the Fourier transform with respect to x and y turns the second-order linear PDE into a second-order linear ODE:

$$\frac{\partial^2 \hat{u}}{\partial t^2} = -v^2 (\kappa_x^2 + \kappa_y^2) \hat{u}$$

Solving the ODE by finding the eigen values gives eigen values of:

$$\lambda = \pm i \sqrt{v^2 (\kappa_x^2 + \kappa_y^2)}$$

With eigen vectors:

$$\hat{u}_1 = \cos \left(t \sqrt{v^2 (\kappa_x^2 + \kappa_y^2)} \right)$$

$$\hat{u}_2 = \sin \left(t \sqrt{v^2 (\kappa_x^2 + \kappa_y^2)} \right)$$

$$\Rightarrow \hat{u}(\kappa_x, \kappa_y) = A(\kappa_x, \kappa_y) \cos \left(vt \sqrt{(\kappa_x^2 + \kappa_y^2)} \right) + B(\kappa_x, \kappa_y) \sin \left(vt \sqrt{(\kappa_x^2 + \kappa_y^2)} \right)$$

Solving the initial conditions:

$$u(x, y, 0) = f(x, y)$$

$$\frac{\partial u}{\partial t} = g(x, y)$$

$$\Rightarrow \hat{u}(\kappa_x, \kappa_y, t) = \hat{f}(\kappa_x, \kappa_y) \cos \left(vt \sqrt{(\kappa_x^2 + \kappa_y^2)} \right) + \frac{\hat{g}(\kappa_x, \kappa_y)}{v \sqrt{(\kappa_x^2 + \kappa_y^2)}} \sin \left(vt \sqrt{(\kappa_x^2 + \kappa_y^2)} \right)$$

Figure 22-25 shows an initial pulse $f(x) = 0$ with velocity $g(x) = e^{\frac{-x^2-y^2}{3}}$ traveling through space. At $T=0$ the plane is completely flat, immediately after a pulse of the aforementioned shape occurs, simulating something like a pebble being dropped into the pool. After the pulse is formed, the wave radiates outward until hitting the boundaries, at which the wave reappears on the opposite side, appearing to bounce back towards the center forming complex wave interference patterns.

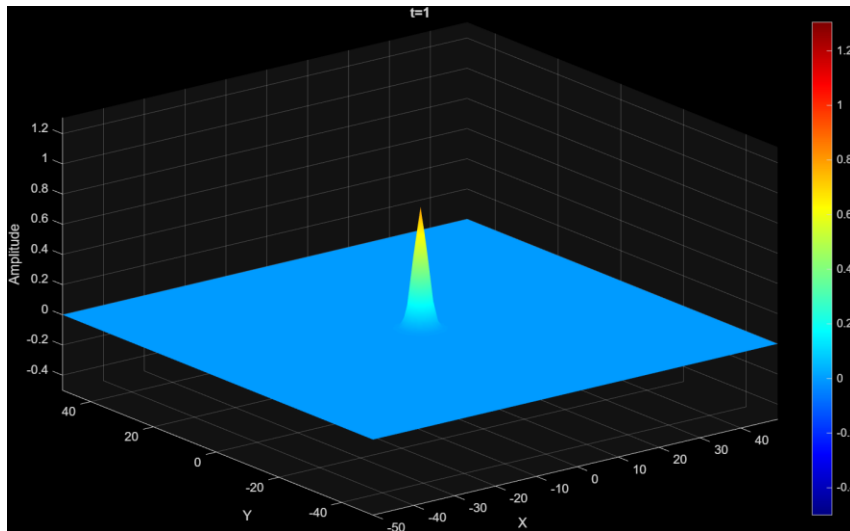


Figure 22. Shows how a 2d gaussian wave with initial velocity = 0 travels across a plane with respect to time at $t=0$.

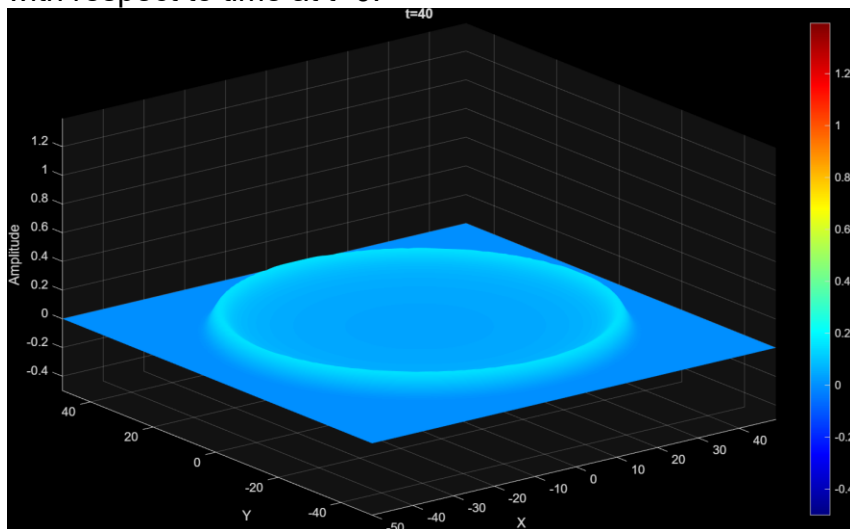


Figure 23. Shows how a 2d gaussian wave with initial velocity = 0 travels across a plane with respect to time at $t=40$.

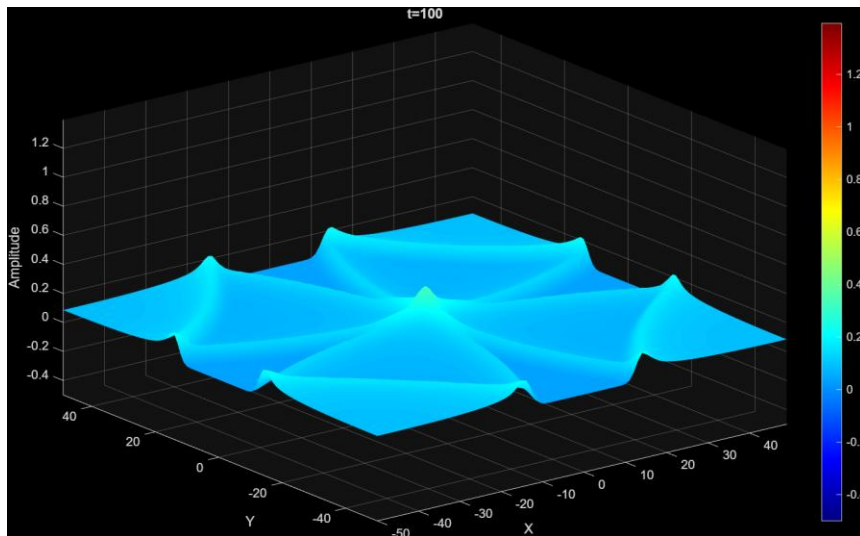


Figure 24. Shows how a 2d gaussian wave with initial velocity = 0 travels across a plane with respect to time at $t=100$.

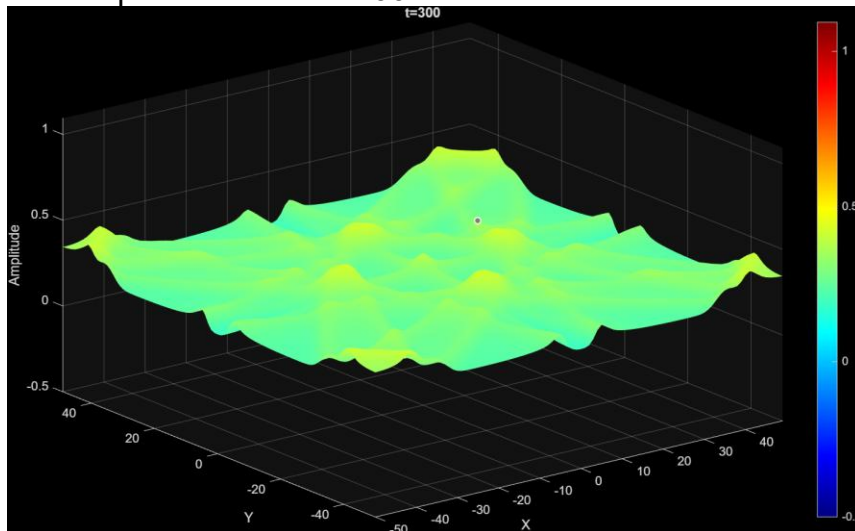


Figure 25. Shows how a 2d gaussian wave with initial velocity = 0 travels across a plane with respect to time at $t=300$.

2D Damped Wave Equation

Language: MATLAB

Theory

This program solves the 2-dimensional wave equation with dampening coefficient of γ :

$$\frac{\partial^2 u}{\partial t^2} + \gamma \frac{\partial u}{\partial t} = v^2 \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right)$$

Taking the Fourier Transform with respect to x and y , turns the PDE into an ODE:

$$\frac{\partial^2 \hat{u}}{\partial t^2} + \gamma \frac{\partial \hat{u}}{\partial t} = -v^2 (\kappa_x^2 + \kappa_y^2) \hat{u}$$

The characteristic polynomial is:

$$\lambda^2 + \gamma \lambda + v^2 (\kappa_x^2 + \kappa_y^2) = 0$$

Attempting to find the eigen values gives:

$$\lambda = \frac{-\gamma \pm \sqrt{\gamma^2 - 4v^2 (\kappa_x^2 + \kappa_y^2)}}{2}$$

When solving with discrete value, κ_x and κ_y are a vector of increasing frequencies.

Therefore, $\sqrt{\gamma^2 - 4v^2 (\kappa_x^2 + \kappa_y^2)}$ has 3 cases:

1. $\gamma^2 - 4v^2 (\kappa_x^2 + \kappa_y^2) < 0$ (Underdamped)
2. $\gamma^2 - 4v^2 (\kappa_x^2 + \kappa_y^2) = 0$ (Critically damped)
3. $\gamma^2 - 4v^2 (\kappa_x^2 + \kappa_y^2) > 0$ (Overdamped)

Solving for the underdamped frequencies

$$\lambda = \frac{-\gamma}{2} \pm \frac{i \sqrt{4v^2 (\kappa_x^2 + \kappa_y^2) - \gamma^2}}{2}$$

$$\text{let: } \omega = \frac{\sqrt{4v^2 (\kappa_x^2 + \kappa_y^2) - \gamma^2}}{2}$$

$$\Rightarrow \hat{u}(\kappa_x, \kappa_y, t) = e^{-\frac{\gamma}{2}t} [A(\kappa_x, \kappa_y) \cos(\omega t) + B(\kappa_x, \kappa_y) \sin(\omega t)]$$

Solving for initial conditions:

$$u(x, y, 0) = f(x, y)$$

$$\frac{\partial u}{\partial t} = g(x, y)$$

$$\Rightarrow \hat{u}(\kappa_x, \kappa_y, t) = e^{-\frac{\gamma}{2}t} \left[\hat{f}(\kappa_x, \kappa_y) \cos(\omega t) + \frac{\hat{g}(\kappa_x, \kappa_y) + \frac{\gamma}{2} \hat{f}(\kappa_x, \kappa_y)}{\omega} \sin(\omega t) \right]$$

Solving for the critically damped frequencies:

$$\{\gamma^2 = 4v^2(\kappa_x^2 + \kappa_y^2)\}$$

$$\lambda = \frac{-\gamma}{2} \pm \frac{\sqrt{0}}{2} = \frac{-\gamma}{2}$$

Eigenvectors:

$$\hat{u}_1 = e^{-\frac{\gamma}{2}t}$$

$$\hat{u}_2 = te^{-\frac{\gamma}{2}t}$$

$$\Rightarrow \hat{u}(\kappa_x, \kappa_y, t) = A(\kappa_x, \kappa_y)e^{-\frac{\gamma}{2}t} + B(\kappa_x, \kappa_y)te^{-\frac{\gamma}{2}t}$$

Solving for initial conditions gives:

$$\hat{u}(\kappa_x, \kappa_y, t) = \hat{f}(\kappa_x, \kappa_y)e^{-\frac{\gamma}{2}t} + [\hat{g}(\kappa_x, \kappa_y) + \frac{\gamma}{2}\hat{f}(\kappa_x, \kappa_y)]te^{-\frac{\gamma}{2}t}$$

Solving for the overdamped frequencies:

$$\{\gamma^2 > 4v^2(\kappa_x^2 + \kappa_y^2)\}$$

$$\lambda = \frac{-\gamma}{2} \pm \frac{\sqrt{\gamma^2 - 4v^2(\kappa_x^2 + \kappa_y^2)}}{2}$$

$$let: \omega = \frac{\sqrt{\gamma^2 - 4v^2(\kappa_x^2 + \kappa_y^2)}}{2}$$

$$\Rightarrow \hat{u}(\kappa_x, \kappa_y, t) = A(\kappa_x, \kappa_y)e^{(-\frac{\gamma}{2} - \omega)t} + B(\kappa_x, \kappa_y)e^{(-\frac{\gamma}{2} - \omega)t}$$

Solving for initial conditions:

$$\begin{aligned} \hat{u}(\kappa_x, \kappa_y, t) = & \left[\hat{f}(\kappa_x, \kappa_y) - \frac{\hat{f}(\kappa_x, \kappa_y)\left(-\frac{\gamma}{2} + \omega\right) - \hat{g}(\kappa_x, \kappa_y)}{2\omega} \right] e^{(-\frac{\gamma}{2} + \omega)t} \\ & + \left[\frac{\hat{f}(\kappa_x, \kappa_y)\left(-\frac{\gamma}{2} + \omega\right) - \hat{g}(\kappa_x, \kappa_y)}{2\omega} \right] e^{(-\frac{\gamma}{2} - \omega)t} \end{aligned}$$

In summary:

1. (underdamped): $\{\gamma^2 < 4v^2(\kappa_x^2 + \kappa_y^2)\}$:

$$\hat{u}(\kappa_x, \kappa_y, t) = e^{-\frac{\gamma}{2}t} \left[\hat{f}(\kappa_x, \kappa_y) \cos(\omega t) + \frac{\hat{g}(\kappa_x, \kappa_y) + \frac{\gamma}{2}\hat{f}(\kappa_x, \kappa_y)}{\omega} \sin(\omega t) \right]$$

$$Where: \omega = \frac{\sqrt{4v^2(\kappa_x^2 + \kappa_y^2) - \gamma^2}}{2}$$

2. (critically damped): $\{\gamma^2 = 4v^2(\kappa_x^2 + \kappa_y^2)\}$:

$$\hat{u}(\kappa_x, \kappa_y, t) = \hat{f}(\kappa_x, \kappa_y)e^{-\frac{\gamma}{2}t} + [\hat{g}(\kappa_x, \kappa_y) + \frac{\gamma}{2}\hat{f}(\kappa_x, \kappa_y)]te^{-\frac{\gamma}{2}t}$$

3. (overdamped): $\{\gamma^2 > 4v^2(\kappa_x^2 + \kappa_y^2)\}$:

$$\hat{u}(\kappa_x, \kappa_y, t) = \left[\hat{f}(\kappa_x, \kappa_y) - \frac{\hat{f}(\kappa_x, \kappa_y) \left(-\frac{\gamma}{2} + \omega\right) - \hat{g}(\kappa_x, \kappa_y)}{2\omega} \right] e^{(-\frac{\gamma}{2} + \omega)t} + \left[\frac{\hat{f}(\kappa_x, \kappa_y) \left(-\frac{\gamma}{2} + \omega\right) - \hat{g}(\kappa_x, \kappa_y)}{2\omega} \right] e^{(-\frac{\gamma}{2} - \omega)t}$$

$$\text{Where: } \omega = \frac{2}{\sqrt{\gamma^2 - 4v^2(\kappa_x^2 + \kappa_y^2)}}$$

Figure 26-29 show a wave with shape $f(x) = -e^{\frac{-x^2-y^2}{3}}$ and velocity $g(x) = 0$ moving through space. Similarly to the regular 2D version, the wave radiates outward until it hits the boundaries. At which the wave reappears on the opposite side, appearing to bounce back towards the center, forming complex wave interference patterns. The key difference is that this equation is damped with $\gamma = 0.01$, meaning that the amplitude of the wave decreases as time increases. It is particularly apparent when comparing amplitudes in Figure 29 from the damped equation to figure 25 of the regular equation.

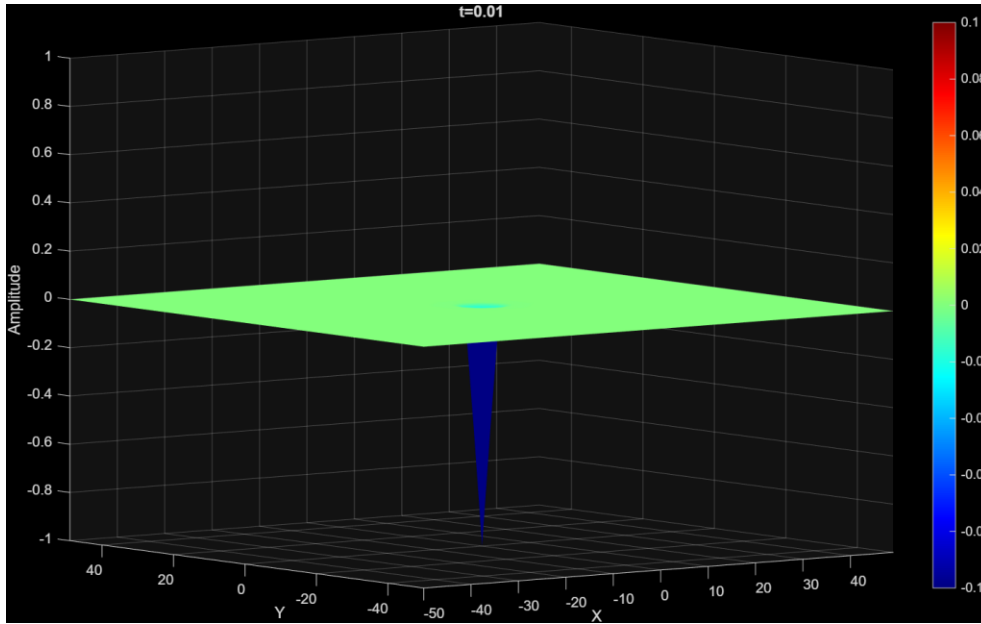


Figure 26. Shows how a 2d gaussian pulse with damping coefficient = 0.01 travels across a plane with respect to time at t=0.

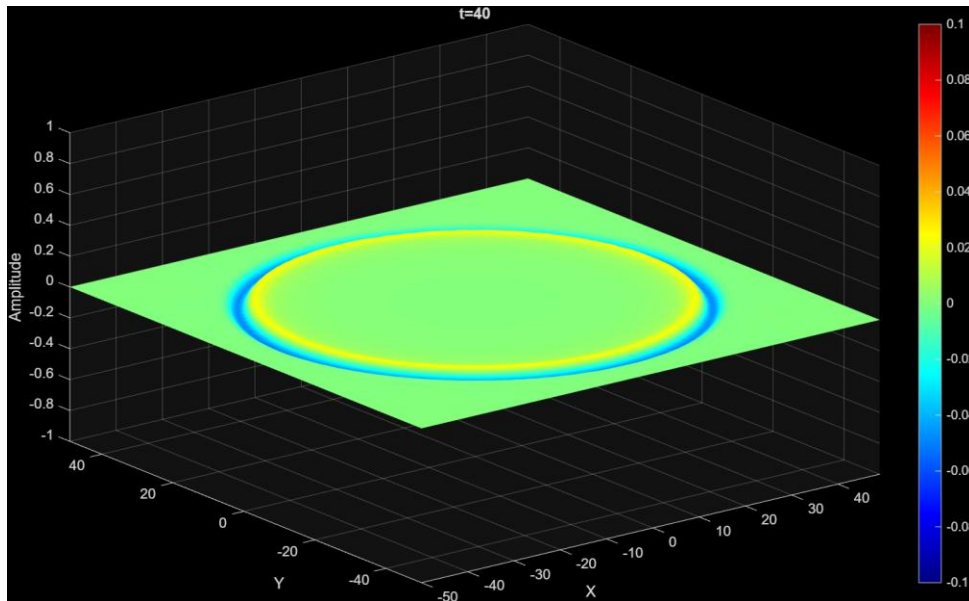


Figure 27. Shows how a 2d gaussian pulse with damping coefficient = 0.01 travels across a plane with respect to time at $t=40$.

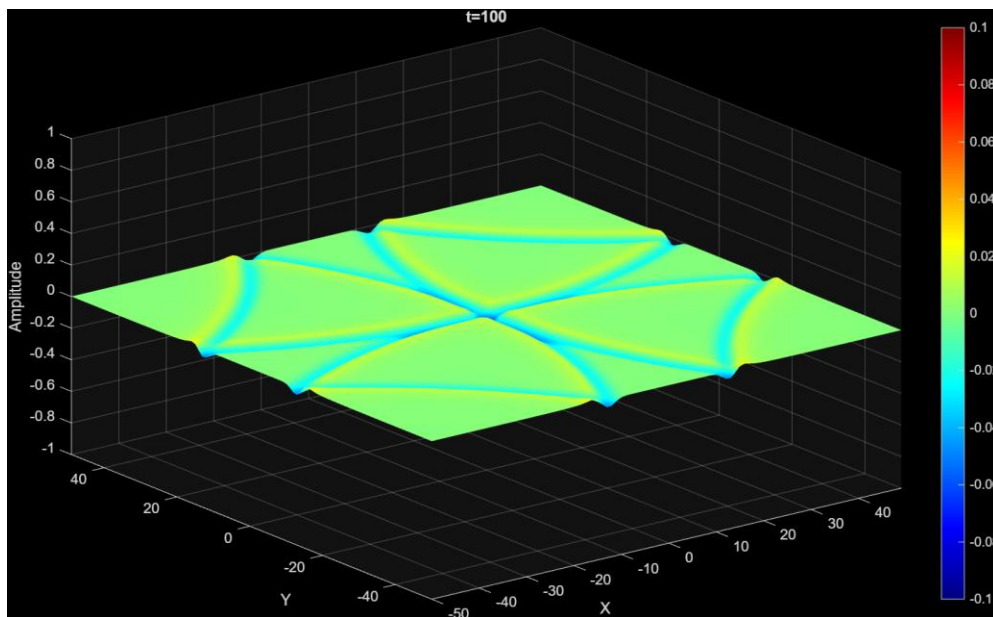


Figure 28. Shows how a 2d gaussian pulse with damping coefficient = 0.01 travels across a plane with respect to time at $t=100$.

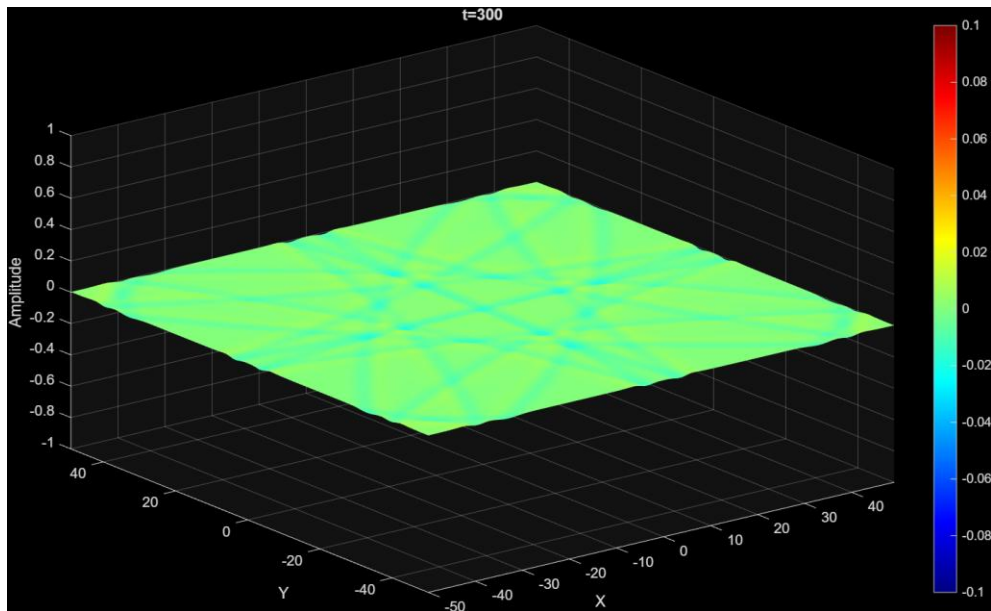


Figure 29. Shows how a 2d gaussian pulse with damping coefficient = 0.01 travels across a plane with respect to time at $t=300$.

Fourier Image Denoising using Low Pass Filter

Language: MATLAB

Theory

This program denoises an image applying a 2D Fourier Transform. Once the image is translated to its spectral domain a low pass filter is applied. The filter works by setting all frequencies higher than a set radius to zero, keeping only the low frequencies. Noise in images are generally made from higher frequencies, thus by cutting all the higher frequencies, the filter effectively eliminates noise by sacrificing detail for denoising by blurring the image.



Figure 30. Original Image.



Figure 31. Denoised Image using 2D FFT.

STFT Image Processing

Language: MATLAB

Theory

This program applies the idea of the Short Time Fourier Transform to 2D datasets. This program takes images and divides it into a series of overlapping 2D windows. The dataset is padded with zeros until a whole number of windows can fit. A windowing function, a 2D gaussian was used in this program, is then applied to each window to ensure that the window is periodic. Since each window overlaps the previous, to keep track of how much each data pointed is windowed by, another 2D matrix is used to store the “window weight” at every point. This window weight matrix will be used to undo the windowing function after reconstruction. The 2D Fourier Transform is applied to each of the windowed datasets, and a low pass filter is used to eliminate the higher frequencies. The inverse transform is then applied, and the dataset is added to the reconstructed matrix. The reconstructed matrix is then divided by window weight matrix in order to undo the window at each point. Lastly, the reconstructed dataset is then trimmed back to the original size.

This program could also be expanded to detect where different textures are located. For example, grass typically has a lot of variation when looking at surrounding pixels, which would correlate to higher frequencies in Fourier space when compared to something like a blank wall. So, the program could output where higher frequencies are located.

The program could also be expanded for edge detection by comparing frequencies located in previous windows.

Another application could be to denoise in specific sections, such as denoising more in the darker portions of the image.

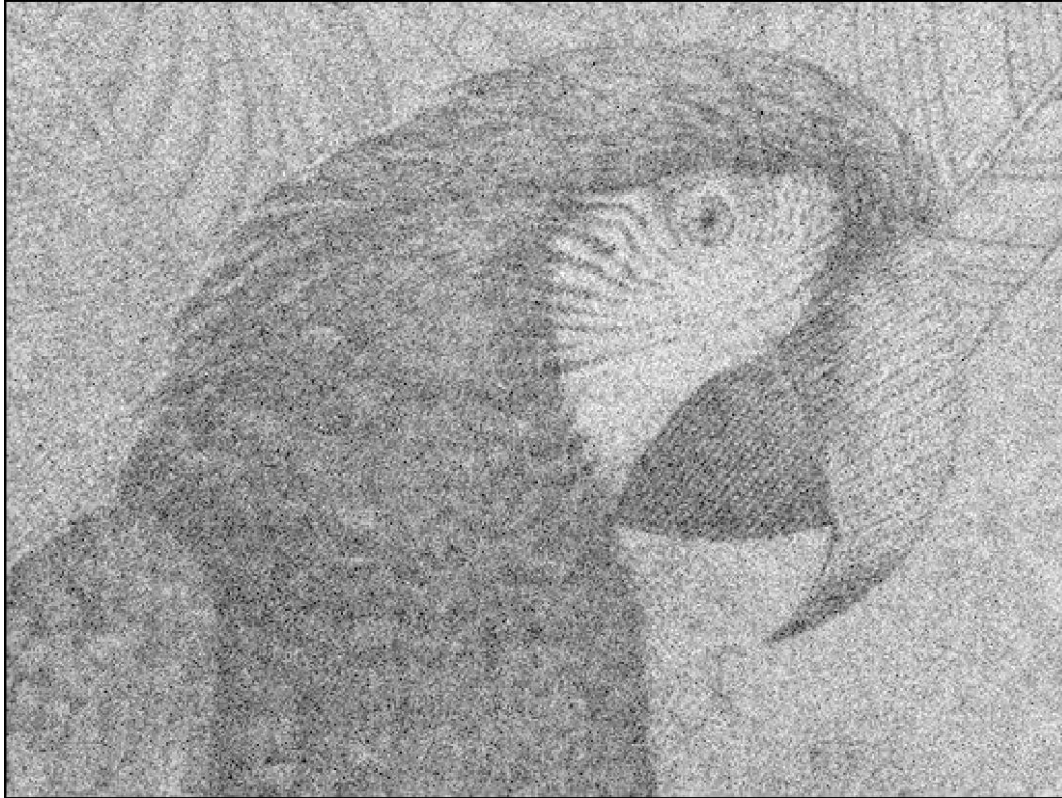


Figure 32. Original Image.



Figure 33. Denoised Image using 2D STFT

MRI Image Processing

Language: MATLAB

Data used in the preparation of this article were obtained from the NYU fastMRI Initiative database (fastmri.med.nyu.edu).

Theory

This program uses real MRI K-space data obtained from the NYU fastMRI Initiative database (fastmri.med.nyu.edu). Before understanding how this program works it would be beneficial to understand the basic idea of how a MRI works.

An MRI works by applying a very strong magnetic field usually 1.5 Tesla or 3.0 Tesla. This magnetic field is strong enough to align the Hydrogen protons within the patient's body. Radio (RF) waves are emitted; these radio waves hit the hydrogen protons exciting the protons causing the nuclei to precess, producing a rotating magnetic field. The protons will eventually return to their equilibriums in a process called relaxation.

Different tissues have different proton densities, meaning the protons precessing in every tissue would contribute different strengths of voltages to the total signal.

To measure the precession frequencies of the hydrogen protons, since each proton has a tiny magnetic field, by applying Lenz's/Faraday's law which states that changes in magnetic field/flux causes an induced current that opposes the change in magnetic flux. By wrapping conductive coils around the MRI machine, the collective oscillation of the hydrogen nuclei will induce a voltage within the coils, which then can be recorded to create a waveform voltage.

To construct the k-space dataset, the MRI machine uses gradients within the magnetic field to encode the x position data and slice location (z position data). When collecting k_x data, also known as the readout phase, a linear magnetic gradient $G_x(x)$ is applied forming a total magnetic field

$$B(x) = B_0 + G_x \cdot x$$

$$B_0 = \text{base magnetic field (1.5T or 3.0T)}$$

$$x = \text{distance along the } x \text{ axis, centered in the middle of the receiving coils}$$

$$G_x = \text{gradient in } X \text{ direction } \left(\frac{T}{m}\right)$$

This gradient change makes it so the frequency at which the Hydrogen nuclei precess becomes dependent on its location such that one side precesses at a slightly higher frequency and the other side precesses at a slightly lower frequency. The angular frequency at which the nuclei process is called the Larmor Frequency, given by this equation:

$$\omega(x) = \gamma(B_0 + G_x \cdot x)$$

$$\gamma = \text{gyromagnetic ratio } \left(\frac{MHz}{T}\right)$$

Therefore, the protons that are located in the high end of the gradient will precess faster than the protons in the low end of the gradient, thus gaining more phase. Meaning, the phase is now a function of space and time, such:

$$\phi(x, t) = \int_0^t \omega(x) dt = \gamma(B_0 + G_x \cdot x)t$$

Ignoring the constant phase shift caused by B_0 :

$$\begin{aligned}\phi(x, t) &= \gamma G_x x \cdot t \\ \text{let: } k_x &= \gamma G_x t \left(\frac{\text{rad}}{m} \right) \\ \Rightarrow \phi(x, t) &= k_x(t) \cdot x\end{aligned}$$

Through Faraday's Law, the rotating transverse magnetization induces an RF voltage within the receiving coil. Therefore, the voltage signal $S(t)$ at time = t is the sum of all the voltages caused by all the spins, or:

$$\begin{aligned}S(t) &= \int \rho(x) e^{-i\phi(x,t)} dx \\ \Rightarrow S(t) &= \int \rho(x) e^{-i\gamma G_x x t} dx\end{aligned}$$

Where $\rho(x)$ is the spin density is, it is essentially the unknown function that describes the distribution of protons through the patient.

Comparing $S(t)$ to the Fourier Transform:

$$S(k_x) = \int \rho(x) e^{-i2\pi k_x x} dx$$

Setting the equations equal gives:

$$\begin{aligned}2\pi k_x &= \gamma G_x t \\ \Rightarrow k_x &= \frac{\gamma G_x t}{2\pi} \left(\frac{\text{cycles}}{m} \right)\end{aligned}$$

In the 2D Fourier Transform, k_y and k_x can be thought of as a gradient of phase shifts, rotating the striped pattern formed by k_x as you move down k_y . So, by sending a pulse of magnetic field gradient through in the y direction, will briefly change the precession frequencies of the hydrogen nuclei, accumulating phase through the duration of the pulse. The phase accumulated will remain after the pulse, thus creating a gradient of phase shifts for the next readout:

$$\begin{aligned}\phi(y) &= \gamma G_y y \Delta t \\ \text{Thus: } k_y &= \gamma G_y \Delta t\end{aligned}$$

For the next readout, the amplitude of G_y will increase and the process will repeat until the last sample.

Therefore, by expanding the previous integral, this function is given:

$$\begin{aligned}S(k_x, k_y) &= \iint \rho(x, y) e^{-i2\pi(k_x x + k_y y)} dx dy = S(t) = \iint \rho(x, y) e^{-i\phi(x, y, t)} dx dy \\ \phi(x, y, t) &= (\omega_y \cdot T_p) + (\omega_x \cdot t) \\ T_p &= \text{Pulse Duration}\end{aligned}$$

By Faraday's law, the procession of the nuclei produces AC voltage. The AC signal at every sample point is recorded using a signal processing technique called Quadrature Detection. Essentially, quadrature detection splits the signal into 'Q' and 'I' components, such that the amplitude of the signal is given by:

$$\|S\| = \sqrt{I^2 + Q^2}$$

And phase shift is given by:

$$\phi = \tan^{-1}\left(\frac{Q}{I}\right)$$

To separate the signal into its Q and I components, it multiplies the signal by a reference cosine signal with frequency of the Larmor frequency, this forms the Q component. Then it multiplies the original signal by a reference sine signal with the same frequency, forming the I component.

In K-space, every sample directly corresponds to a k_x with the Q and I components of the voltage at that point forming the complex Fourier coefficients, with Q being the imaginary component and I being the real component.

Finally, for the z axis (slice selection), another constant magnetic gradient is applied in the z direction. Then the machine selects an RF pulse that will only excite the Larmor Frequencies within the desired slice's range, effectively ignoring the other parts of the z gradient. To select the frequency of the RF pulse would follow an equation similar to:

$$f(z) = \gamma(B_0 + G_z z)$$

This works because the x and y gradient are insignificant compared to the z gradient, so the RF pulse should be able to excite all protons within the desired range.

Essentially, an MRI is an attempt at doing a physical 2D Fourier Transform, turning the object into its spatial frequencies. To do this it fixes the k_y frequencies and then sees how much each k_x frequency is contributing, which is encoded within the voltage waveform using a magnetic gradient. For the next row of k_y it changes the k_y and then repeats finding how much each k_x frequency is contributing to the total voltage signal but this time at a set k_y frequency.

The process of taking an MRI would be as follows. First, a strong constant magnetic field B_0 aligns the proton spins. To select a slice, a magnetic field gradient is applied in the z direction. While this gradient is applied, an RF pulse with a limited bandwidth centered around the desired Larmor frequency is transmitted. Because the magnetic field strength varies with z position, only protons within the desired slice are excited. After the protons are excited, a brief magnetic gradient called the phase-encoding gradient is applied in the y direction. After every readout, the phase-encoding gradient increases its amplitude slightly.

Next, the readout phase, a x gradient is applied during this phase. The precessing protons create an AC voltage which is then recorded. Quadrature detection is used to split the signal into real and imaginary components. Each time sample of the AC voltage signal maps along the k_x in one row. One complete readout forms one row of k-space. The process is repeated with slightly different y phase-encoding gradients, forming the next row of k-space data.

For example, for a 100×100 image, 100 readouts are required, each containing 100 samples, resulting in 10,000 total k-space samples.

Once all the k-space data is collected a 2D Inverse Fast Fourier Transform reconstructs the final image. The process can then be repeated for each slice.

(Note: This has not been verified by an expert, inaccuracies may exist)

This program uses H5read to read data from the FastMRI downloaded .tar file. H5read locates the data group 'kspace.' The kspace group contains two 3D matrixes, one is the real component of the frequencies, the other is the imaginary components. Two the dimensions are the k_x and k_y . The third dimension is the slices. By simply applying the inverse Fourier transform, the MRI image can be constructed.

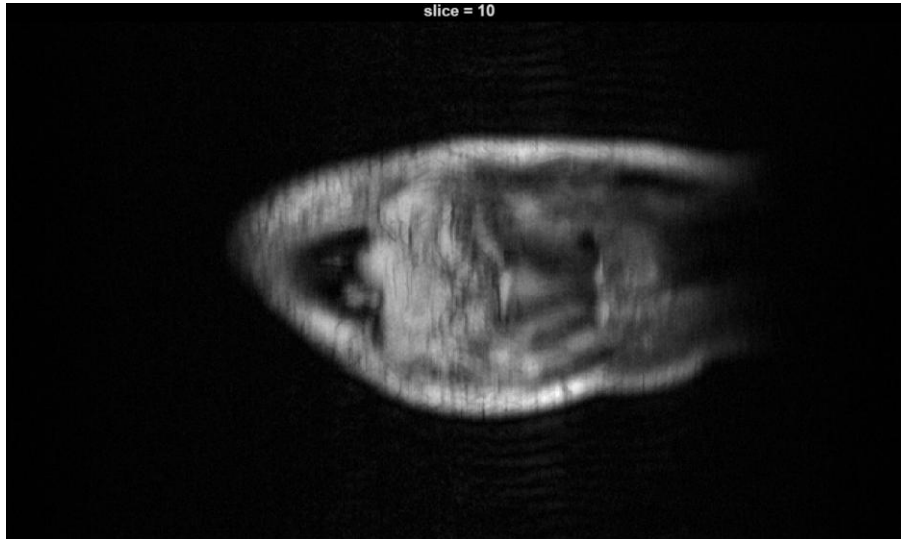


Figure 34. 1st slice of an MRI of a patient's knee.



Figure 35. 20th slice of an MRI of a patient's knee.

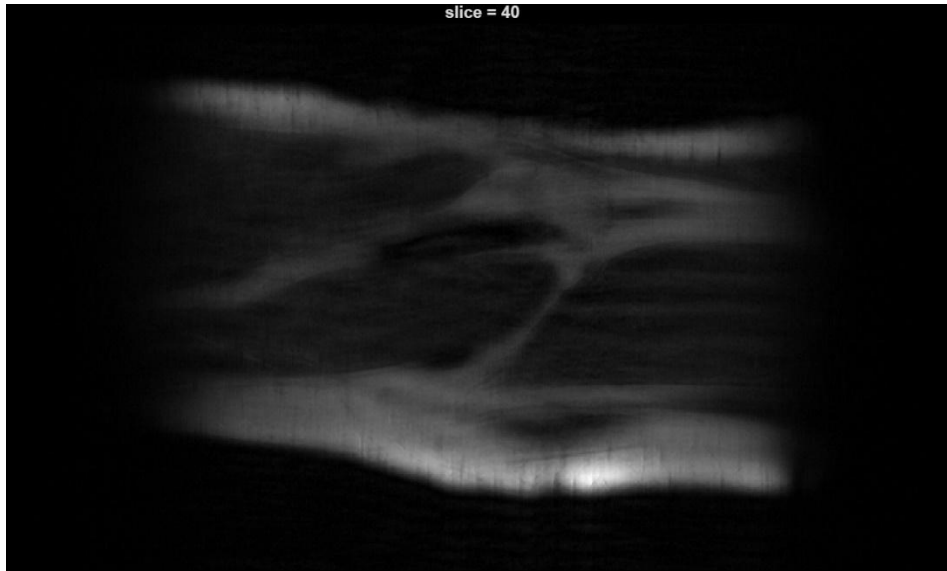


Figure 36. Top (40th) slice of an MRI of a patient's knee.

Simulated MRI

Language: MATLAB

Theory

This program takes an image and essentially resizes the image using similar concepts to an MRI in an attempt to simulate an MRI. Instead of using physical tissues, this program uses the brightness values of an image in replacement of the unknown function rho that describes proton distribution in a typically MRI.

To simulate the MRI processes, this program makes a matrix of the same size as the image and assigns a digital magnetic field, with the x gradient added to every pixel. Then using the Larmor Frequency equation with the Gyromagnetic ratio for hydrogen:

$$\omega(x) = \gamma B_x$$
$$\gamma = 2\pi \times 42.577 \times 10^6 \left(\frac{\text{rad}}{\text{s}} \right)$$
$$B = \text{magnetic field}$$

It calculates the Larmor frequency for every pixel. Then it calculates the collective voltage signal produced by all the Larmor frequencies to achieve the desired x-resolution, the number of time samples must be equal to the resolution. So, the total sample time would be calculated by $T = \frac{xRes}{\text{Sample Rate}}$. To calculate the simulated voltage, signal the program uses the discrete form of the following formula:

$$S(t) = \iint \rho(x, y) e^{-i\phi(x, y, t)} dx dy$$
$$\phi(x, y, t) = (\omega_y \cdot T_p) + (\omega_x \cdot t)$$
$$T_p = \text{Pulse Duration}$$

Using the calculated signal, it applies Quadrature detection to turn the real signal into complex values. It assigns that correlated vector to one row of k-space. Then it repeats with the next phase-encoding gradient until the desired y-resolution is achieved. Then an Inverse Fast Fourier Transform is used to reconstruct the image.



Figure 37. Original image with resolution of 512x512.



Figure 38. Resultant image with resolution of 50x50.

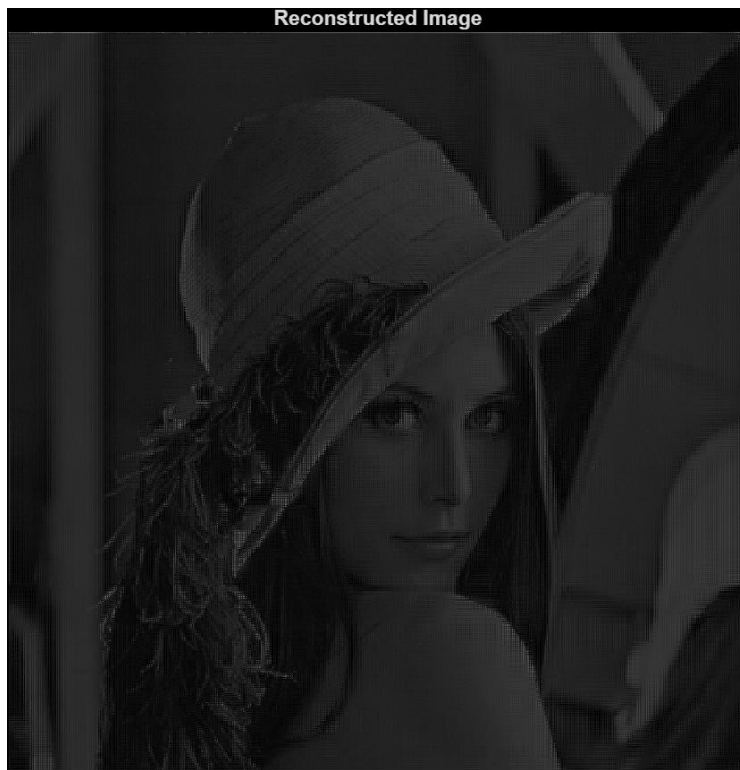


Figure 39. Resultant image with resolution 1000x1000.

Discussion

Across all these projects the idea remains the same; take a signal and convert it into the spectral domain. From there several different processes can be performed, like cutting off unwanted frequencies or displaying what frequencies are most prominent.

The STFT was an important part of these projects, it allowed to bridge the gap between the time domain and spectral domain. The FFT gave the ability to go from 100% time resolution to 100% frequency resolution. The STFT allowed a midpoint between spectral and time, where there was some resolution in both frequency domain and time domain. However, the STFT had several drawbacks, mainly with the fixed window size, it often caused spectral leakage and other artifacts that decreased accuracy. This is where something like the Wavelet Transform offers an advantage.

The Wavelet Transform is a newer method developed to do similar processes to the Fourier Transform. Wavelets are most similar to the STFT, as it takes a mother wavelet usually denoted as $\psi(t)$ which is a small, localized signal, as illustrated in figure 40, and sets a certain frequency and compares that wavelet across the entire signal using the convolution. The Wavelet Transform not only tell where the frequency occurs but also when, much like the STFT.

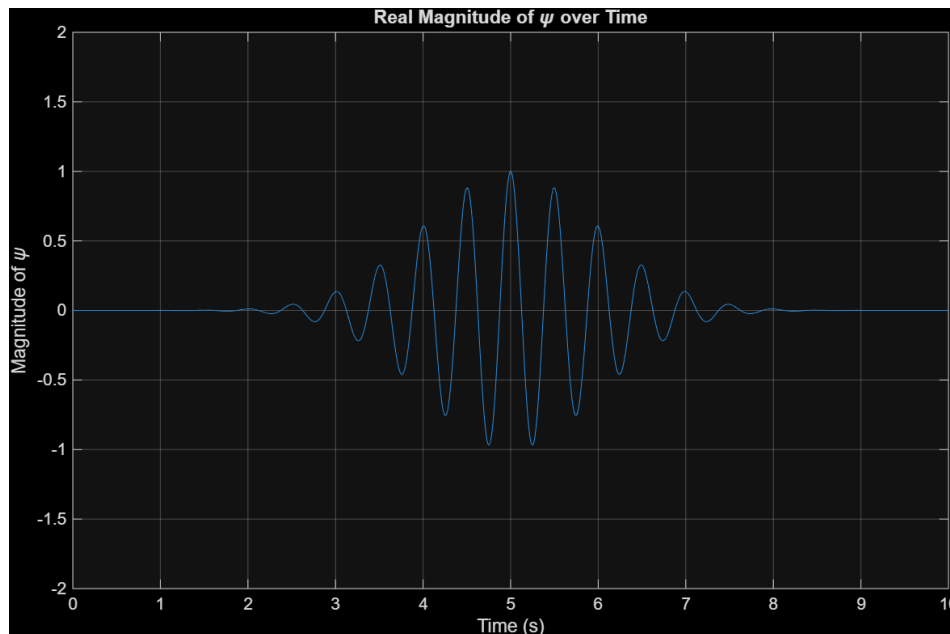


Figure 40. Graph of the Morlet Wavelet $\psi(t) = e^{i\omega t}e^{-t^2/2}$, developed by Jean Morlet in the 1980s, commonly used in physics.

There are many more methods, like the Hilbert Transform, Principal Component Analysis, Proper Orthogonal Decomposition, and Empirical Mode Decomposition, just to name a couple. However, all of them keep idea; take a signal and decompose it into its frequency components. All of them have their advantages and disadvantages to each other, the exact comparisons and applications is discussion for another project. It is

worth noting that none of them are meant as a replacement for each other, just different tools for whatever application they are a part of.

There are many more applications of the Fourier Transform that were not discussed nor made into a project in this paper. The projects listed in this paper gave a broad overview of some applications. One notable application that was only briefly touched upon is data compression. By transforming data into the frequency domain and discarding the least significant frequency components, the data can be stored in a much smaller form. Reconstructing from this reduced spectral data introduces some loss, this process is a form of lossy compression. Another closely related transform, the Discrete Cosine Transform (DCT), is the basis for compression standards like JPEG. The DCT is much like the 2D STFT project made in this paper, however it only uses the real components (cosine components) to save of size, it also does not use overlapping windows like the 2D STFT.

Continuing off the idea of MRI machines, a critical piece of technology used in chemistry is the Nuclear Magnetic Resonance (NMR) machine. It essentially applies the idea as a MRI machine, by applying a constant strong magnetic field around a thin vile of a fluid and sending RF pulses to excite the hydrogen (or other element). Due to valence electrons surrounding, this also known as shielding, the hydrogen atoms, depending of the molecule the hydrogens will precess at a different frequency. A RF receiver coil is used to read an AC voltage resulting from the precessing magnetic fields. A FFT is applied to the signal to construct a PSD, each frequency spike corresponds to how much a certain molecule is present in the sample. The PSD can be averaged overtime to eliminate noise.

A future real-world application project of this could be crude NMR machine. The basic idea would be to use strong neodymium magnetics to construct a consistent magnetic field of ~ 0.5 Telsa. A simple RF pulse generator could be constructed, and copper wire wrapped around the thin vile to receive the AC signal. The wire would likely have to be connected to a signal resonator to amplify the AC signal. A FFT can applied and averaged overtime to construct the PSD graph. The signal would likely be far to weak and noisy to produce any meaningful data, however it should be enough to demonstrate the concept of the NMR. Again, this just is a concept, the project is based on my current understanding and speculation. More research would be required, but it is a project that I would like to try to make in the future.

Conclusion

This collection of projects has proven that the Fourier Transform is incredibly diverse in its applications. Originally developed to solve heat transfer equations, it has expanded to nearly every field of math and physics. With the development of the DFT and FFT, it has expanded even further into signal processing, image processing, data compression, enabling lifesaving technology like the MRI and critical scientific tools like the NMR machine, and many more applications. Newer more efficient methods like wavelets have been developed, but even still the Fourier Transform remains at the core of it, and still plays a critical role in science, math, medicine, and computer science.

Looking back, these projects started with a simple simulated data denoiser and ended with reconstructing MRI images from an actual medical database. It has been an unique and extremely interesting experience, especially considering I had no real interest in math or coding a year ago. I accredit my professors at Arizona Western College for that. In the future, I hope to continue to expand my knowledge of the Fourier Transform and its related fields, and I hope to put it to some actual value.

References

- [1] S. L. Brunton, "Fourier Analysis," *YouTube playlist*. [Online]. Available: https://youtube.com/playlist?list=PLMrJAKhleNNT_Xh3Oy0Y4LTj0Oxo8GqsC. [Accessed: May 27, 2026].
- [2] M. Newman, "How to Implement the FFT: A Coding Tutorial." Jul. 03, 2025. Available: <https://youtu.be/isUMxOlpcj4?si=K6iwdvvrN2wxuTPG>
- [3] A. D. Elster and C. Emeritus, "Questions and Answers In MRI," *MRIquestions*. <https://mriquestions.com/index.html>
- [4] F. Knoll, K. Hammernik, C. Zhang, S. Moeller, T. Pock, D. K. Sodickson, and M. Akçakaya, "Deep-learning methods for parallel magnetic resonance image reconstruction: A survey of the current approaches, trends, and issues," *Radiology: Artificial Intelligence*, vol. 2, no. 1, p. e190007, Jan. 2020, doi: 10.1148/ryai.2020190007.
- [5] J. Zbontar, F. Knoll, A. Sriram, T. Murrell, Z. Huang, M. J. Muckley, A. Defazio, R. Stern, P. Johnson, M. Bruno, *et al.*, "fastMRI: An Open Dataset and Benchmarks for Accelerated MRI," *arXiv preprint* arXiv:1811.08839, Nov. 2018.